

**Санкт-Петербургский государственный электротехнический университет
“ЛЭТИ” им. В. И. Ульянова (Ленина)**

(СПбГЭТУ “ЛЭТИ”)

Направление подготовки: 09.03.01 “Информатика и вычислительная техника”
Профиль: “Вычислительные машины, комплексы, системы и сети”

**Факультет компьютерных технологий и информатики
Кафедра вычислительной техники**

К защите допустить:

Заведующий кафедрой

д. т. н., профессор

_____ М. С. Куприянов

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

Тема: “Модули сопряжения эмулятора радара”

Студент

_____ Д. М. Соболев

Руководитель

к. т. н., доцент

_____ О. И. Буренева

Консультант по экономическому
обоснованию

к. э. н., доцент

_____ И. М. Антонова

Консультант от кафедры

к. т. н., доцент, с. н. с.

_____ И. С. Зуев

Санкт-Петербург
2020 г.

Санкт-Петербургский государственный электротехнический университет
“ЛЭТИ” им. В. И. Ульянова (Ленина)

(СПбГЭТУ “ЛЭТИ”)

Направление 09.03.01 “Информатика и
вычислительная техника”

Профиль “Вычислительные машины, комплексы,
системы и сети”

Факультет компьютерных технологий
и информатики

Кафедра вычислительной техники

УТВЕРЖДАЮ

Заведующий кафедрой ВТ

д. т. н., профессор

(М. С. Куприянов)

“___” _____ 202__ г.

ЗАДАНИЕ

на выпускную квалификационную работу

Студент Соболев Даниил Михайлович

Группа № 6305

1. Тема Модули сопряжения эмулятора радара

(утверждена приказом № _____ от _____)

Место выполнения ВКР: Санкт-Петербургский государственный
электротехнический университет им. В.И.Ульянова(Ленина)

- 2. Объект исследования** Комплексная система обнаружения и обработки сигналов радара.
- 3. Предмет исследования** Модули сопряжения эмулятора системы радара.
- 3. Цель** Разработка модулей сопряжения с интерфейсами Ethernet и Link
- 4. Исходные данные** Описание системы эмулятора радара, технические описания интерфейсов Ethernet и Link.
- 5. Содержание** Обзор структуры сопряжения эмулятора системы радара, разработка модуля сопряжения с Ethernet, разработка модуля сопряжения с Link с использованием языков проектирования аппаратуры Verilog/VHDL.
- 6. Технические требования** Частота работы модуля сопряжения Ethernet – не менее 50 МГц, модуля сопряжения с Link – не менее 100 МГц.
- 7. Дополнительные разделы** Экономическое обоснование выпускной квалификационной работы.
- 8. Результаты** Разработаны модули сопряжения с интерфейсами Ethernet и Link, проведено моделирование работы обоих модулей и синтез обоих

модулей, модуль сопряжения с Ethernet также проверен экспериментально на макетной плате. Отчетные материалы: пояснительная записка, реферат, аннотация, презентация.

Дата выдачи задания
«_____» _____ 202__ г.

Дата представления ВКР к защите
«_____» _____ 202__ г.

Руководитель
к. т. н., доцент

О. И. Буренева

Студент

Д. М. Соболев

**Санкт-Петербургский государственный электротехнический университет
“ЛЭТИ” им. В. И. Ульянова (Ленина)
(СПбГЭТУ “ЛЭТИ”)**

Направление 09.03.01 “Информатика и
вычислительная техника”
Профиль “Вычислительные машины, комплексы,
системы и сети”
Факультет компьютерных технологий
и информатики
Кафедра вычислительной техники

УТВЕРЖДАЮ
Заведующий кафедрой ВТ
д. т. н., профессор
(М. С. Куприянов)
“___” _____ 202__ г.

**КАЛЕНДАРНЫЙ ПЛАН
выполнения выпускной квалификационной работы**

Тема Модули сопряжения эмулятора радара

Студент Соболев Даниил Михайлович

Группа № 6305

№ этапа	Наименование работ	Срок выполнения
1	Обзор литературы по теме работы	10.03 – 17.03
2	Анализ протоколов интерфейсов	17.03 – 24.03
3	Анализ и выбор способов реализации модулей сопряжения	24.03 – 01.04
4	Разработка модулей сопряжения с использованием языков проектирования аппаратуры	01.04 – 20.04
5	Моделирование и тестирование модулей	20.04 – 03.05
6	Экспериментальная проверка модуля сопряжения с Ethernet	03.05 – 11.05
7	Технико-экономическое обоснование работы	11.05 – 14.05
8	Оформление пояснительной записки	14.05 – 01.06
9	Предварительное рассмотрение работы	02.06
10	Представление работы к защите	02.06

Руководитель

к.т.н., доцент

Студент

_____ О. И. Буренева

_____ Д. М. Соболев

РЕФЕРАТ

Пояснительная записка содержит: 74 с., 36 рис., 9 табл., 6 приложений, 22 источника литературы.

Цель работы: разработка модулей сопряжения эмулятора радара с внешними интерфейсами Ethernet и Link.

В выпускной квалификационной работе приводится обзор системы сопряжения эмулятора радара, а также анализ реализуемых интерфейсов и способов их реализации.

В результате работы с помощью языков проектирования аппаратуры разработаны и отмоделированы модули сопряжения с интерфейсами Ethernet и Link. Модуль сопряжения с Ethernet также проверен на макетной плате.

Результат работы в перспективе будет использоваться как часть комплексной системы обнаружения и обработки сигналов автомобильного радара.

ABSTRACT

The objective of the final work is the development of interface modules for the radar emulator for interfaces Ethernet and Link.

A review of the structure of the radar emulator is presented in the work as well as an analysis of interfaces implemented in the modules and ways of implementing said interfaces.

As for the results of the work, the modules have been successfully developed and tested in a modelling environment, the Ethernet interface module also has been tested on a real board.

In the long-term, the modules will be used as a part of a complex system of signal detection and processing for an automotive radar.

СОДЕРЖАНИЕ

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ.....	9
ВВЕДЕНИЕ.....	10
1 ОБЗОР СТРУКТУРЫ СОПРЯЖЕНИЯ ЭМУЛЯТОРА СИСТЕМЫ ОБРАБОТКИ СИГНАЛОВ РАДАРА	12
1.1 Общие сведения о системе	12
1.2 Интерфейс Ethernet	15
1.2.1 Канальный уровень	15
1.2.2 Транспортный уровень	16
1.2.3 Выбор способа разработки модуля сопряжения с Ethernet.....	17
1.3 Интерфейс Link	18
1.3.1 Дифференциальная передача сигналов.....	18
1.3.2 Стандарт LVDS	20
1.3.3 Протокол интерфейса Link.....	21
1.3.4 Выбор способа разработки модуля сопряжения	22
1.3.5 Выводы по главе.....	23
2 РАЗРАБОТКА МОДУЛЯ СОПРЯЖЕНИЯ С ИНТЕРФЕЙСОМ ETHERNET	24
2.1 Описание функционирования модуля сопряжения с Ethernet.....	24
2.1.1 Описание ARM-сервера.....	24
2.1.2 Работа с памятью на чипе в FPGA	25
2.1.3 Условия тестирования модулей первичной обработки	26
2.2 Описание программы.....	26
2.2.1 Соединение FPGA- и ARM-частей между собой и блоками памяти.....	26
2.2.2 Программа сервера на HPS	27
2.2.3 Модули на FPGA	28
2.3 Моделирование FPGA-подмодулей	31
2.3.1 Пример 1 – 8-разрядный инвертор	32
2.3.2 Пример 2 – Пороговое устройство	33
2.4 Синтез FPGA-модулей.....	36
2.5 Эксперимент на макетной плате.....	37
2.5.1 Ход эксперимента	37
2.5.2 Результаты эксперимента	40
3 РАЗРАБОТКА МОДУЛЯ СОПРЯЖЕНИЯ С ИНТЕРФЕЙСОМ LINK	42
3.1 Описание функционирования модуля сопряжения с Link	42
3.2 Описание программы.....	43
3.2.1 Модуль приемника Link-порта	43
3.2.2 Модуль передатчика Link-порта.....	44
3.2.3 Устройства обработки данных.....	45
3.3 Моделирование процесса приема, обработки и передачи	47
3.4 Синтез и расположение контактов	48
3.5 Выводы по главе.....	49
4 ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ МОДУЛЕЙ СОПРЯЖЕНИЯ	50
4.1 Составление плана-графика выполнения работ	50
4.2 Расчет затрат на оплату труда исполнителей	51

4.2.1 Расчет основной заработной платы исполнителей	51
4.2.2 Расчет дополнительной заработной платы	52
4.2.3 Расчет обязательных социальных отчислений.....	52
4.2.4 Итоговые затраты на оплату труда.....	53
4.3 Расчет затрат на приобретение материалов и спецоборудования.....	53
4.4 Расчет амортизационных отчислений.....	54
4.5 Расчет накладных расходов.....	55
4.6 Расчет сметной стоимости разработки.....	55
4.7 Выводы по главе.....	56
ЗАКЛЮЧЕНИЕ	57
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	58
ПРИЛОЖЕНИЕ А. Система HPS-SoC в Platform Designer	60
ПРИЛОЖЕНИЕ Б. Код программы ARM-сервера.....	62
ПРИЛОЖЕНИЕ В. RTL-диаграммы подмодулей модулей сопряжения	65
ПРИЛОЖЕНИЕ Г. Подробные результаты моделирования модуля сопряжения с Link	68
ПРИЛОЖЕНИЕ Д. Назначение контактов модуля сопряжения с Link	71
ПРИЛОЖЕНИЕ Е. RTL-диаграммы модуля сопряжения с Link	72

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

ARM – RISC-архитектура процессоров, используемых в системах на кристалле.

ПЛИС – программируемая логическая интегральная схема.

ДТП – дорожно-транспортное происшествие.

FPGA – Field Programmable Gate Array, – конфигурируемое полупроводниковое устройство, разновидность ПЛИС.

SoC – System on Chip, система на кристалле.

HPS – Hard Processor System, часть SoC, отведенная под ARM-процессор.

UDP – User Datagram Protocol, протокол Интернета.

RTL – Register Transfer Level, уровень разработки цифровых интегральных схем, на котором схема рассматривается с точки зрения логических операций, применяемых к данным при их передаче между регистрами.

DSP – Digital Signal Processor, процессор цифровой обработки сигналов.

LVDS – Low-Voltage Differential Signaling, низковольтная дифференциальная передача сигналов.

PLL – Phase-Locked Loop, система фазовой автоподстройки частоты, позволяющая, помимо прочего, преобразовывать входной опорный тактовый сигнал путем умножения, деления и фазового сдвига.

PC – Personal Computer, персональный компьютер.

TCP – Transmission Control Protocol, протокол Интернета.

ЭМП – электромагнитное поле.

Testbench – программа, предназначенная для тестирования модулей, написанных на языке проектирования аппаратуры.

ЯПА – язык проектирования аппаратуры.

ВВЕДЕНИЕ

Дороги по сей день остаются местом повышенной опасности. В 2019 году на территории Российской Федерации произошло более 160 тысяч ДТП, в результате которых погибло почти 17 тысяч человек [1], на общемировом уровне дорожно-транспортные происшествия входят в верхнюю десятку причин смерти [2]. По данным агентства NHTSA, причиной около 94% ДТП является человеческий фактор [3].

Для уменьшения травматизма в результате ДТП с середины XX века применяются средства пассивной безопасности, такие как ремни безопасности и подушки безопасности. В настоящее время автопроизводители разрабатывают и внедряют средства активной безопасности, то есть средства, вмешивающиеся в управление автомобилем с целью предотвращения ДТП. Важнейшими компонентами в системах активной безопасности авто являются средства обнаружения препятствий, такие как радары, лидары и видеокамеры.

Принцип действия радара заключается в считывании отражений радиоволн для обнаружения объектов; по сравнению с лидарами и видеокамерами, радары более надежны, их работа не зависит от погодных условий, а также не предоставляют высоких требований к средствам обработки данных [4]. Радары уже применяются в системах адаптивного круиз-контроля, а также в системах сбора данных для автономных автомобилей.

Подобные системы, помимо устройств обнаружения, состоят из множества блоков и компонентов первичной обработки данных, которые нуждаются в проверке и тестировании. Для решения этих задач разрабатывается эмулятор среды тестирования блоков первичной обработки сигналов. Данная работа посвящена разработке модулей сопряжения эмулятора с интерфейсами внешних устройств.

Целью данной работы является разработка модулей сопряжения с интерфейсами Ethernet и Link для обеспечения эмулятора среды тестирования связью с внешними устройствами, такими как PC или DSP.

Объектом данной работы является комплексная система приема и обработки сигналов с радара. Предметом разработки являются модули сопряжения эмулятора.

Задачи:

1. Анализ вариантов реализации и протоколов приема-передачи данных интерфейсов Ethernet и Link.

2. Разработка модулей сопряжения.

3. Моделирование, тестирование модулей.

4. При наличии возможности экспериментальная проверка на ПЛИС.

Обзор системы сопряжения эмулятора, описание интерфейсов, а также анализ способов реализации модулей сопряжения приведены в первой главе. Во второй главе приведено описание разработки модуля сопряжения с интерфейсом Ethernet. В третьей главе приведено описание разработки модуля сопряжения с интерфейсом Link. В четвертой главе приводится технико-экономическое обоснование выпускной работы. Заключение включает основные выводы по работе.

1 ОБЗОР СТРУКТУРЫ СОПРЯЖЕНИЯ ЭМУЛЯТОРА СИСТЕМЫ ОБРАБОТКИ СИГНАЛОВ РАДАРА

Данная глава посвящена обзору структуры сопряжения эмулятора системы радара, частью которой являются модули сопряжения, анализу принципов действия и протоколов интерфейсов сопряжения, а также выбору способа разработки модулей.

1.1 Общие сведения о системе

Общая схема комплексной системы радара приведена на рисунке 1.1.

Главными компонентами системы являются:

1. Радарная решетка (RTX).
2. Тракт первичной обработки.
3. Модули сопряжения.
4. Два цифровых сигнальных процессора (DSP).
5. Оперативная память (RAM).

Эмулятор радара представляет собой программно-аппаратную систему, состоящую из приложения, реализованного на PC, и аппаратной системы на базе FPGA, реализующей первичную обработку. Главной задачей эмулятора радара является тестирование модулей тракта первичной обработки, как индивидуальное, так и в составе цепочки. Приложение на PC представляет собой генератор тестовых воздействий для тракта первичной обработки и разрабатывается с применением языков высокого уровня.

Чтобы передавать данные от приложения на PC в тракт первичной обработки на FPGA и отправлять результаты обработки обратно приложению, нужно наладить канал связи через Ethernet, для чего и разрабатывается соответствующий модуль сопряжения. После прохождения тракта первичной обработки данные должны передаваться на DSP, также на стороне FPGA связь с DSP, производящим вторичную обработку, должен иметь диспетчер внешних интерфейсов.

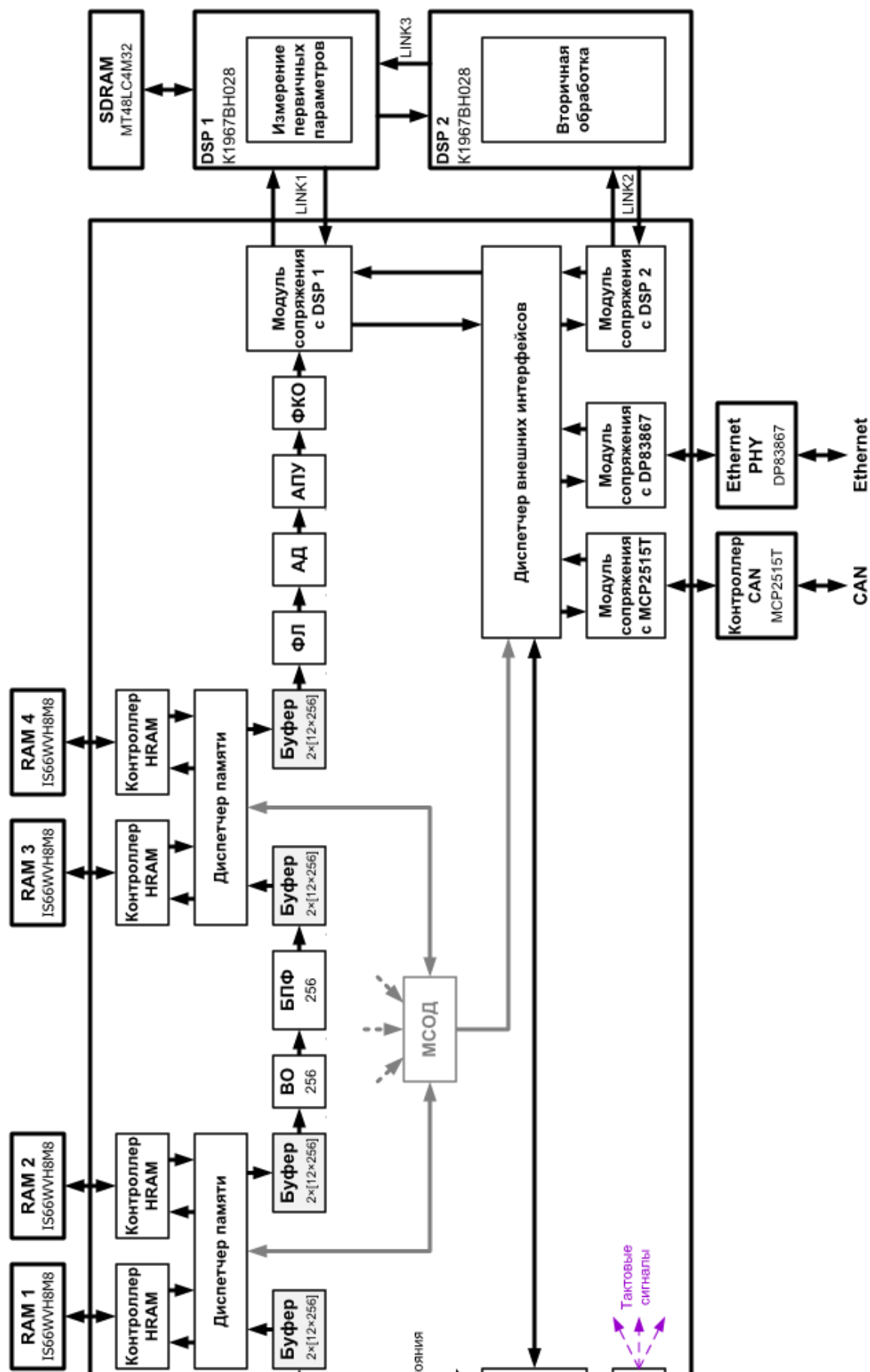
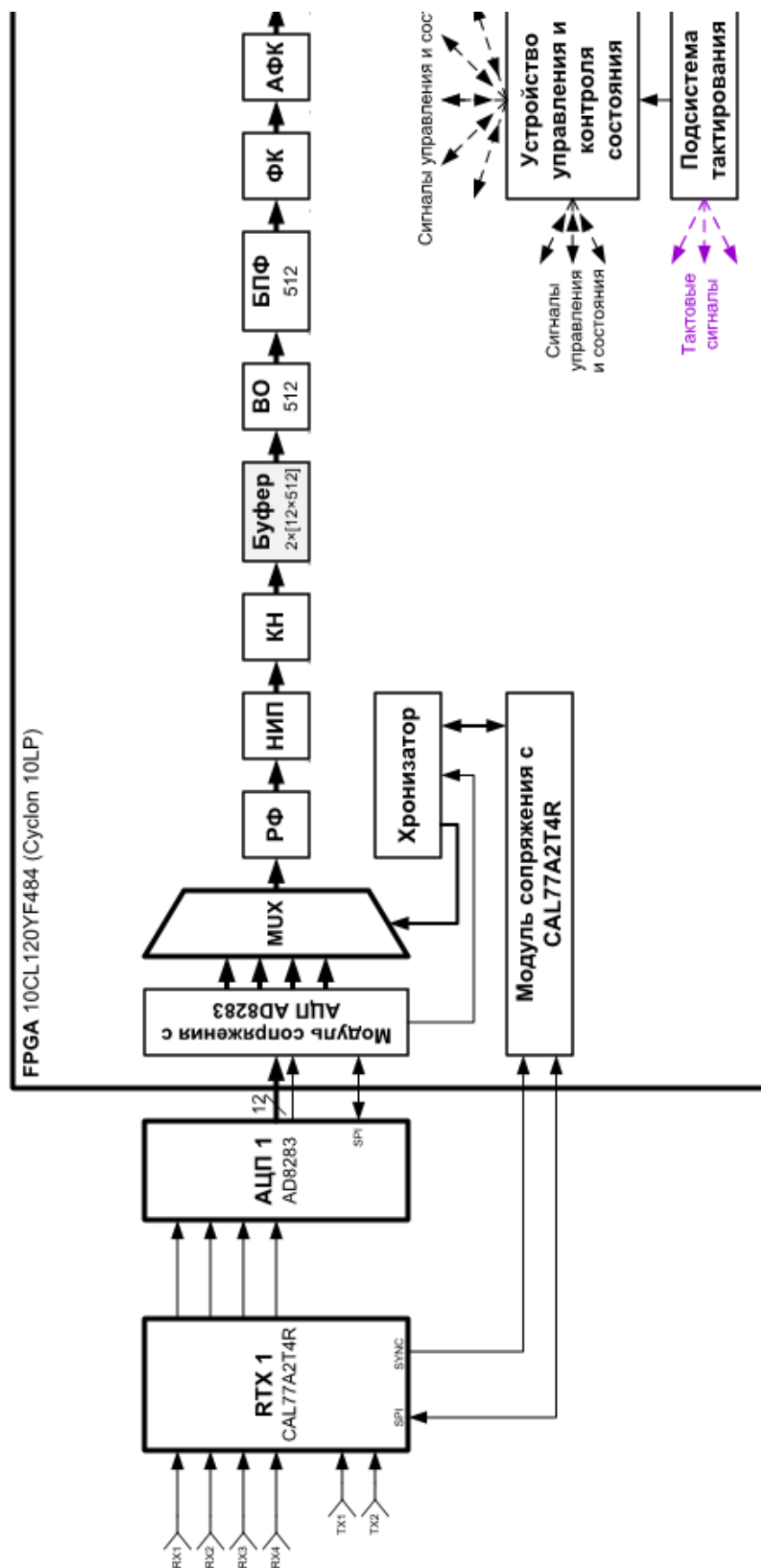


Рисунок 1.1 – Общая схема системы обработки сигналов с радара



Продолжение рисунка 1.1

Для того, чтобы обеспечить связь FPGA с DSP, разрабатывается модуль сопряжения с интерфейсом Link.

1.2 Интерфейс Ethernet

Канал связи через порт Ethernet можно реализовать несколькими способами с применением различных протоколов.

1.2.1 Канальный уровень

Один из вариантов реализации связи через Ethernet – использовать базовый функционал, предоставляемый спецификацией Ethernet, а именно передавать данные на канальном уровне модели OSI с использованием Ethernet-фреймов.

Формат Ethernet-фрейма [5] представлен в таблице 1.1.

Таблица 1.1 – Ethernet-фрейм

Преам-була	НРК	Адрес получателя	Адрес отправителя	Тип/длина	Данные	КПК
------------	-----	------------------	-------------------	-----------	--------	-----

Составляющие Ethernet-фрейма:

1. Преамбула (7 байт) – информирует принимающую сторону о начале нового фрейма.
2. НРК (начало разделителя кадра) (1 байт) – сигнализирует о начале нового кадра.
3. Адрес получателя (6 байт) – адрес устройства назначения.
4. Адрес отправителя (6 байт) – адрес устройства отправления.
5. Тип/длина (2 байта) – определяет длину кадра (если значение меньше или равно десятичному числу 1500) или тип протокола инкапсуляции данных (если значение превышает десятичное число 1536).
6. Данные (46-1500 байт) – содержит данные для передачи.
7. КПК (контрольная последовательность кадра) (4 байта) – конец фрейма, используется для обнаружения ошибок в кадре.

Отдельного рассмотрения требует адресация устройств в Ethernet [6]. Каждое устройство обладает уникальным 48-битным идентификатором,

называемым Media Access Control Address (MAC-адрес). MAC-адреса присваиваются устройствам заводом-производителем. Формат MAC-адреса представлен в таблице 1.2.

Таблица 1.2 – Формат MAC-адреса

Октет	0	1	2	3	4	5
Двоичный вид	00110101	01111011	00010010	00000000	00000000	00000001
Шестнадцатеричный вид	AC	DE	48	00	00	80

Двоичный вид в таблице 1.2 отражает порядок передачи MAC-адреса по сети, каждый октет передается, начиная с младшего разряда.

Подобный протокол передачи поддается реализации на FPGA (например, с использованием FPGA-ядер для Ethernet компании Intel [7]), однако данный способ отличается повышенной трудоемкостью и длительностью разработки не только на стороне FPGA, но и на стороне PC, поскольку не используются протоколы Интернета, следовательно, повышается сложность установления канала связи между FPGA и PC. Еще одним недостатком данной реализации является необходимость перепрошивки FPGA каждый раз, когда нужно изменить адрес устройства, с которого принимаются данные, чего не требуется при использовании протоколов транспортного уровня.

1.2.2 Транспортный уровень

Другой путь реализации канала связи через Ethernet-порт – использовать протоколы Интернета на транспортном уровне, такие как TCP [8] и UDP [9], позволяющие использование 32-битных IPv4-адресов для идентификации отправителя и получателя. Протокол TCP обеспечивает надежность доставки, в отличие от UDP, однако, TCP существенно сложнее, и для задачи передачи небольшого количества данных целесообразнее использовать более простой UDP.

Использование протоколов транспортного уровня упрощает задачу налаживания канала связи со стороны РС, реализовать такой канал на стороне FPGA радара можно путем построения системы на кристалле. Системы на кристалле, как правило, реализуются двумя способами:

- целиком в FPGA с использованием процессора Nios II [10];
- HPS-SoC: система на кристалле с отдельным ARM-процессором [11];

Nios II – софт-процессор компании Intel, поддающийся широкой конфигурации. Сконфигурированный процессор затем генерируется в виде файлов на языке проектирования аппаратуры и имплементируется в FPGA [12]. Nios II поддерживает запуск встроенной операционной системы Linux, на которой можно запускать написанные на C программы и которая поддерживает протоколы Интернета, то есть взаимодействие с Ethernet можно описать без использования языков проектирования аппаратуры. Однако, Nios II – софт-процессор, поэтому на его имплементацию тратятся существенные ресурсы FPGA, которых впоследствии может не хватить на реализацию прочих модулей, нуждающихся в тестировании.

Система с отдельным ARM-процессором также поддерживает запуск системы Linux и протоколы Интернета, однако, в отличие от системы на Nios II, не занимает ресурсы FPGA. Однако для реализации подобной системы нужно обладать доступом к соответствующей отладочной плате, в то время как Nios II поддерживается на целом ряде FPGA-устройств.

1.2.3 Выбор способа разработки модуля сопряжения с Ethernet

Таким образом, для решения задачи установления канала связи через Ethernet существует три способа:

1. Разработка модуля сопряжения на канальном уровне с использованием базового протокола Ethernet: способ время- и трудозатратен, причем как с точки зрения разработки на FPGA, так и на РС.

2. Разработка модуля сопряжения на транспортном уровне с использованием UDP с помощью системы на кристалле: гораздо менее трудоемка с обеих сторон сопряжения, может быть реализована с помощью:

- Nios II: софт-процессор, занимает ресурсы FPGA, поддерживается широким набором FPGA-устройств;
- HPS-SoC: ARM-процессор, не занимает ресурсы FPGA, требуется соответствующая отладочная плата;

Для решения поставленной задачи решено, что наиболее целесообразным решением является разработка модуля сопряжения на канальном уровне с использованием протокола UDP, реализованная с помощью системы на кристалле типа HPS-SoC, поскольку доступна плата данной конфигурации, что дает возможность освободить ресурсы FPGA от задачи создания канала связи через Ethernet. В этом случае на стороне FPGA потребуется написание модулей для работы с общей с HPS памятью.

1.3 Интерфейс Link

Передача данных через интерфейс Link осуществляется по стандарту LVDS, регламентирующему технологию низковольтной дифференциальной передачи сигналов. В рамках этого подраздела рассмотрены особенности дифференциальной передачи и стандарта LVDS, а также протокола передачи данных через сам интерфейс Link.

1.3.1 Дифференциальная передача сигналов

Технология дифференциальной передачи сигналов [13, 14] основана на применении двух комплементарных, то есть одинаковых по модулю напряжения, но противоположных по полярности, сигнальных линий (*дифференциальной пары*) для организации одной линии передачи данных. Общее схематическое изображение дифференциальной линии передачи представлено на рисунке 1.2. [14]

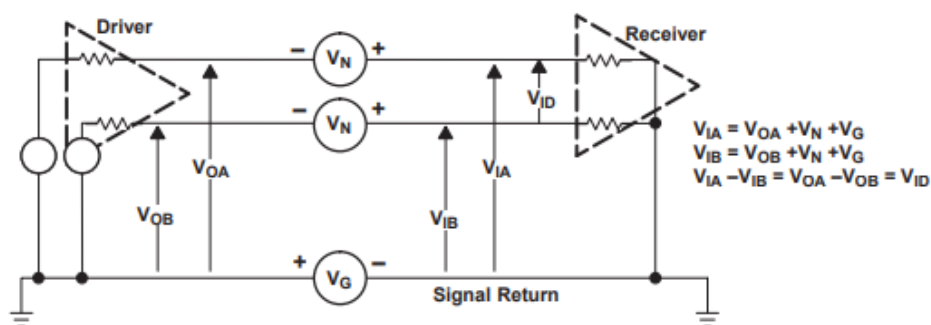


Рисунок 1.2 – Дифференциальная передача сигнала

Логический выход линии данных определяется через разницу напряжений на сигнальных линиях.

Поскольку дифференциальная передача требует две сигнальные линии для представления одной линии данных, ее реализация сложнее и дороже реализации несимметричной передачи, когда требуется всего одна сигнальная линия. Но преимуществ у дифференциальной передачи гораздо больше [13]:

1. Главное преимущество дифференциальной передачи – высокая помехоустойчивость. Поскольку используется два проводника, которые, во-первых, постоянно находятся под напряжением, а во-вторых, это напряжение одинаково по модулю и противоположно по знаку, при появлении внешней помехи она затрагивает оба сигнала в равной степени. Поскольку приемник реагирует на разницу напряжений между сигнальными линиями, на логическом выходе внешняя помеха не сказывается.

2. Еще одно преимущество дифференциальной передачи – малая генерация собственных помех. Каждая сигнальная линия при переключениях сигнала создает собственные ЭМП помех, при этом создаваемые ЭМП очень близки по амплитуде, но противоположны по полярности, соответственно, при условии близкого расположения сигнальных линий (например, при использовании витой пары), помехи от сигнальных линий будут взаимно гасить друг друга.

3. Вследствие высокой помехоустойчивости дифференциальная передача может работать на малых напряжениях, что позволяет использовать

низковольтные источники питания и, соответственно, уменьшить потребляемую мощность устройства и повысить рабочие частоты.

Передача данных с помощью дифференциальной пары на электрическом уровне сейчас широко распространена в высокоскоростных интерфейсах передачи данных.

1.3.2 Стандарт LVDS

LVDS (Low-Voltage Differential Signaling) – стандарт, регламентирующий технологию высокоскоростной низковольтной дифференциальной передачи данных [15]. Схема типичной LVDS-пары «передатчик-приемник» представлена на рисунке 1.3.

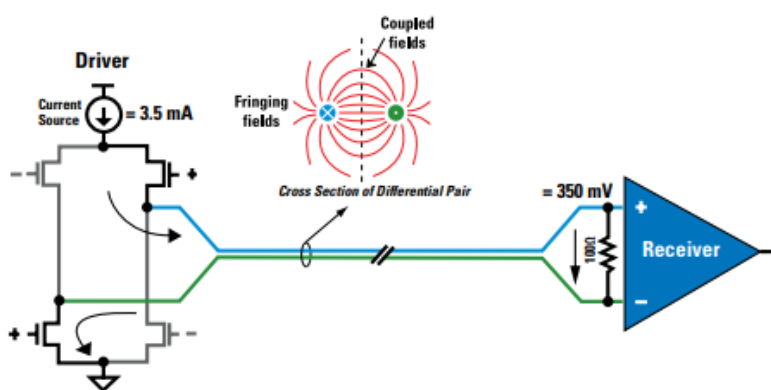


Рисунок 1.3 – Пара «передатчик-приемник» в LVDS

В передатчике находится источник тока номиналом 3.5 мА, который по первой сигнальной линии дифференциальной пары протекает к резистору номиналом 100 Ом, на котором, в свою очередь, появляется напряжение в 350 мВ (*дифференциальное напряжение*), после чего ток по второй сигнальной линии дифференциальной пары возвращается на линию земли. При прямом направлении тока на выходе приемника появляется логическая единица, при обратном – логический ноль.

Регламентированные стандартом LVDS значения напряжений [16]:

- высокий выходной уровень напряжения передатчика: максимум 1,602 В, минимум 1,249 В;

- низкий выходной уровень напряжения передатчика: максимум 1,148 В, минимум 1,002 В;

- дифференциальное выходное напряжение передатчика: максимум 454 мВ, минимум 247 мВ;
- напряжение смещения передатчика: максимум 1,375 В, минимум 1,125 В;
- выходное напряжение смещения приемника: максимум 2,35 В, минимум 0,5 В;
- порог чувствительности приемника: <100 мВ;

Обеспечение высокой чувствительности приемника в его диапазоне напряжений обеспечивает хорошую помехоустойчивость и устойчивость к синфазным сдвигам между приемником и передатчиком [15].

Благодаря своим характеристикам LVDS обеспечивает высокую скорость передачи данных при низком энергопотреблении, что играет ключевую роль в таких задачах как обмен данными между DSP и другими устройствами. Для этих целей и существует интерфейс Link.

1.3.3 Протокол интерфейса Link

Интерфейс Link предназначен для передачи данных как между несколькими DSP, так и между DSP и внешними устройствами через порты связи. В рамках данной работы поставлена задача разработать модуль сопряжения с Link, на другой стороне которого стоит DSP 1967BH028 производства фирмы «Миландр» [17].

Порты связи DSP представляют собой пары «передатчик-приемник» с отдельными LVDS-шинами приема и передачи данных. Разрядность каждой шины – 4 бита, присутствует поддержка однобитного режима передачи данных. Конфигурация Link-порта представлена на рисунке 1.4.

Порты связи поддерживают двойную скорость передачи данных, то есть передачу данных на обоих фронтах тактового сигнала. Данные передаются в формате 128-битных квадрослов. Иллюстрация принципа передачи данных представлена на рисунке 1.5 [18]. Описание выводов Link-порта представлено

в таблице 1.3 [17]. На линии LVDS P подается истинное значение сигнала, на линии LVDS N – противоположное.

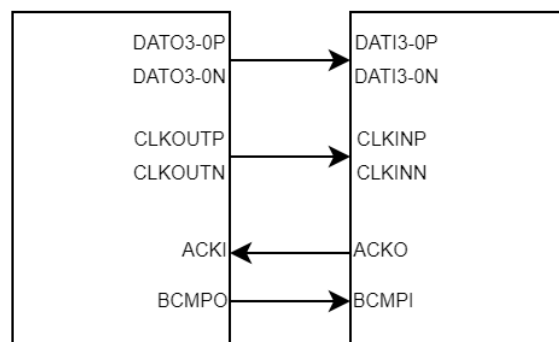


Рисунок 1.4 – Link-порт, конфигурация связи между приемником (справа) и передатчиком

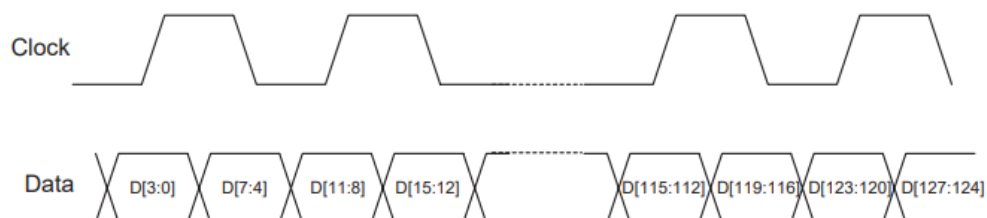


Рисунок 1.5 – Принцип передачи данных порта связи

Таблица 1.3 – Link, описание выводов

Сигнал	Описание
DATO3-0P	Шина данных передачи, LVDS P
DATO3-0N	Шина данных передачи, LVDS N
CLKOUTP	Тактовый генератор передачи, LVDS P
CLKOUTN	Тактовый генератор передачи, LVDS N
ACKI	Входной сигнал подтверждения передатчика
BCMPO	Выходной сигнал завершения блока передатчика
DATI3-0P	Шина данных приема, LVDS P
DATI3-0N	Шина данных приема, LVDS N
CLKINP	Тактовый генератор приема, LVDS P
CLKINN	Тактовый генератор приема, LVDS N
ACKO	Выходной сигнал подтверждения приемника
BCMPI	Входной сигнал завершения блока приемника

С помощью сигнала ACKO приемник указывает передатчику, принимающему сигнал ACKI, что можно продолжать передачу. С помощью сигналов BCMPO передатчик информирует приемник, принимающий сигнал BCMPI, что передача блока данных завершена.

1.3.4 Выбор способа разработки модуля сопряжения

На стороне FPGA требуется реализовать приемник и передатчик данных Link-порта, а также устройства работы с этими данными.

Поскольку модуль сопряжения с Link используется только для связи DSP и FPGA и не предполагается его подключение к внешним устройствам, реализовывать его с применением системы на кристалле в рамках работы нецелесообразно. Следовательно, путей разработки остается немного:

1. Разработать составные части модуля сопряжения с нуля. Возможный вариант такой реализации – разработать приемник и передатчик с помощью модулей LVDS компании Intel. [19] Также для реализации приемника и передатчика нужно разработать схему для поддержки двойной передачи данных. Данный способ достаточно время- и трудозатратен.

2. Взять шаблонные реализации приемника и передатчика Link-порта. В этом случае нужно разработать только устройства работы с данными, а также разобрать и протестировать работу шаблонных приемника и передатчика.

Поскольку в наличии имелись шаблонные модули для Link-порта, отвечающие требованиям по протоколу интерфейса, было принято решение разработать модуль сопряжения с использованием шаблонных реализаций.

1.3.5 Выводы по главе

В результате выполнения обзора системы эмулятора и анализа реализуемых интерфейсов передачи были избраны методы разработки каждого модуля (для Ethernet – разработка модуля сопряжения на транспортном уровне с использованием протокола UDP с помощью системы на кристалле типа HPS-SoC, для Link – разработка модуля сопряжения с применением шаблонных модулей приемника и передатчика). Сформулированы конкретные действия, которые требуется произвести в рамках разработки:

- Ethernet: написание UDP-сервера на языке C и модулей работы с памятью на FPGA;
- Link: написание устройств работы с данными, подключение шаблонных модулей;

2 РАЗРАБОТКА МОДУЛЯ СОПРЯЖЕНИЯ С ИНТЕРФЕЙСОМ ETHERNET

Данная глава посвящена разработке модуля сопряжения с интерфейсом Ethernet, описанию алгоритмов функционирования модуля и его реализации.

2.1 Описание функционирования модуля сопряжения с Ethernet

Модуль сопряжения с Ethernet задействует ARM-компонент и FPGA-компонент SoC, а также память на чипе (On-Chip Memory), через которую FPGA- и ARM-компоненты взаимодействуют. Память на чипе разделена на четыре блока: блок для служебной информации, два блока для входных данных, блок для выходных данных.

Общая схема модуля сопряжения представлена на рисунке 2.1.

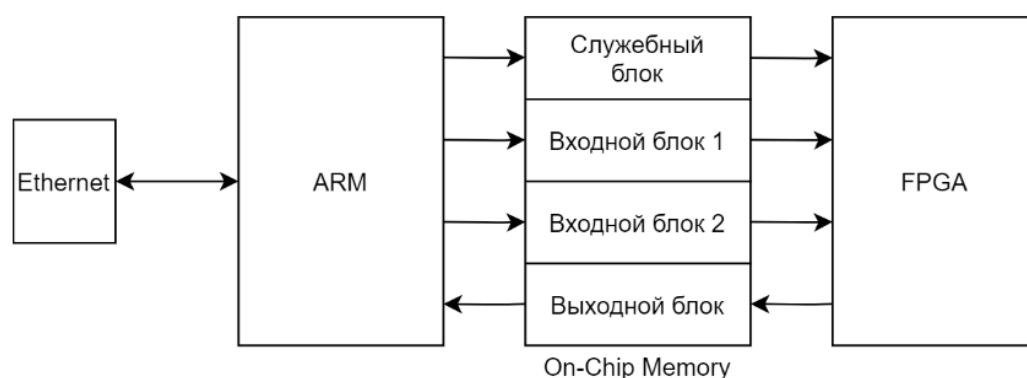


Рисунок 2.1 – Общая схема модуля сопряжения с Ethernet

На стороне ARM запущен сервер, который принимает с порта и отправляет в порт датаграммы, соответствующие протоколу UDP, а также работает с памятью на чипе. На стороне FPGA происходит выгрузка данных из памяти на чипе в тестируемую цепочку модулей и загрузка в память результатов тестирования.

2.1.1 Описание ARM-сервера

Сервер, запущенный в HPS-части системы на кристалле, выполняет два сценария: прием с порта и передачу в FPGA служебной информации и входных данных; прием полученных в FPGA в результате тестирования данных и передачу их в порт.

Алгоритм для операции приема:

1. Принять от клиента датаграмму, содержащую служебную информацию (служебная датаграмма).
2. Принять датаграмму, содержащую данные (датаграмма данных).
3. В случае успешного приема датаграмм записать служебную информацию и данные в соответствующие блоки памяти на чипе, в противном случае завершить работу.
4. Перейти в режим ожидания готовности выходных данных.

Алгоритм для операции передачи:

1. По сигналу готовности выходных данных сформировать из данных в выходном блоке памяти на чипе датаграмму выходных данных.
2. Отправить датаграмму выходных данных клиенту.
3. В случае успешной отправки датаграммы перейти в режим ожидания новой служебной датаграммы, в противном случае завершить работу.

Служебная датаграмма содержит следующую информацию:

1. Количество каналов радара.
2. Номер первого блока в цепочке.
3. Номер последнего блока в цепочке.
4. Количество входных данных.

На данном этапе наибольший интерес представляет количество входных данных.

2.1.2 Работа с памятью на чипе в FPGA

Поскольку всё взаимодействие с Ethernet происходит в ARM-части, в FPGA-части нужно только работать с памятью на чипе, выгружая и загружая данные в соответствующие блоки. Для успешной операции сопряжения нужно выполнить три операции:

1. По сигналу готовности входных данных (успешный прием и выгрузка служебной датаграммы и датаграммы входных данных) выгрузить и обработать информацию из блока служебной информации.

2. Выгружать и передавать на вход цепочки модулей первичной обработки входные данные из одного из блоков входных данных.

3. В зависимости от готовности последнего модуля цепочки выдавать данные загружать результат работы цепочки в блок выходных данных.

2.1.3 Условия тестирования модулей первичной обработки

Если нужно протестировать такой модуль, что выдает выходное значение практически сразу после получения входных данных, то выгрузка данных из памяти и загрузка выходных значений должна происходить параллельно. Для модулей, имеющих внутренние буферы памяти, загрузка выходных значений в память происходит по сигналу готовности выходных данных.

Например, если нужно проверить какую-нибудь комбинационную схему, допустим, n -разрядный инвертор, то выгрузка входных данных и загрузка выходных начинаются параллельно по сигналу завершения считывания служебной информации.

Если же нужно проверить модуль, имеющий внутренний буфер памяти, например, пороговое устройство, загрузка выходных данных начнется по готовности их выдачи со стороны модуля, в случае этого конкретного устройства – после последнего значения на входе.

2.2 Описание программы

Далее приведены описания программ ARM-части и FPGA-части, а также описание их соединения друг с другом.

2.2.1 Соединение FPGA- и ARM-частей между собой и блоками памяти

Для соединения компонентов модуля сопряжения между собой используется средство Platform Designer программы Quartus Prime Lite. В нем же объявляются блоки памяти на чипе.

Размерности блоков памяти на чипе следующие:

- блок служебной информации: 5 однобайтных слов.
- блоки входных данных: по 256 двухбайтных слов в каждом.

- блок выходных данных: 512 четырехбайтных слов.

Увеличение разрядности выходных данных в 2 раза обусловлено допустимостью операции умножения на выходах некоторых модулей цепочки.

Доступ ARM-компонента и FPGA-компонента к одним и тем же блокам памяти достигается путем объявления двухпортовой памяти, тактируемой одним тактовым сигналом, и выделения ARM- и FPGA-компонентам отдельных портов. Соединение компонентов показано на рисунке 2.2.

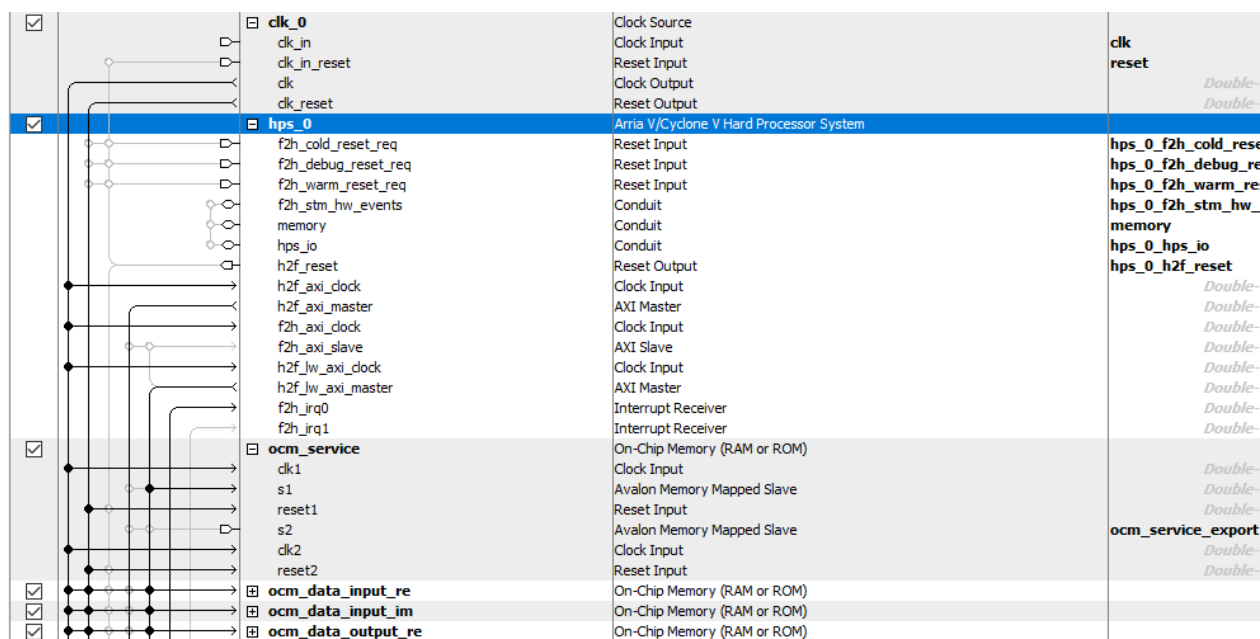


Рисунок 2.2 – Соединение компонентов в Platform Designer

Здесь *hps_0* – ARM-компонент, *ocm_service*, *ocm_data_input_re*, *ocm_data_input_im*, *ocm_data_output_re* – блоки памяти. Один порт блоков памяти соединяется с *hps_0*, другой экспортируется для использования в FPGA. Блокам памяти назначаются базовые адреса, впоследствии используемые в ARM-сервере.

Полная схема соединения компонентов в Platform Designer приведена в приложении А.

2.2.2 Программа сервера на HPS

Сервер написан с помощью языка C с использованием Linux-библиотеки *socket*, а также с использованием API для HPS компании Altera.

Форматы датаграмм, принимаемых от клиента, представлены на рисунках 2.3 (служебная датаграмма) и 2.4 (датаграмма данных).

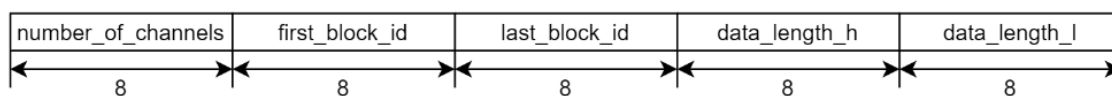


Рисунок 2.3 – Формат служебной датаграммы

Здесь *number_of_channels* – количество каналов радара, *first_block_id* – номер первого блока цепочки, *last_block_id* – номер последнего блока цепочки, *data_length_h* – старший байт количества данных, *data_length_l* – младший байт количества данных. Необходимость хранить количество данных в двух байтах обусловлена размерностью входных данных – максимум 512 отсчетов на канал, число 512 в один байт не умещается.



Рисунок 2.4 – Формат датаграммы данных

Здесь *data_re* – данные для первого блока памяти, *data_im* – данные для второго блока памяти. Всего датаграмма содержит до 256 четырехбайтных слов, каждое из которых затем делится на два двухбайтных слова и записывается в соответствующий блок памяти.

Также сервер формирует и отправляет датаграмму выходных данных, которая содержит до 512 четырехбайтных слов.

Код программы сервера приведен в приложении Б.

2.2.3 Модули на FPGA

Для взаимодействия с памятью на чипе со стороны FPGA в соответствии со структурным подходом к проектированию разработаны три отдельных подмодуля: подмодуль для выгрузки служебной информации (*eth_service*), подмодуль для выгрузки и передачи входных данных в модули цепочки первичной обработки (*eth_read*), подмодуль для загрузки выходных данных цепочки первичной обработки в память на чипе (*eth_write*). Разработка модулей велась с помощью ЯПА Verilog. Выбор именно этого языка для

описания модулей обусловлен прошлым знакомством с ним в рамках учебного плана, а также более простым, чем у альтернатив, написанием testbench. Оформление производилось в соответствии с правилами документации проектов для ПЛИС [20] и требованиями по единой системе программной документации. [21, 22]

Схематические обозначения подмодулей представлены на рисунке 2.5.

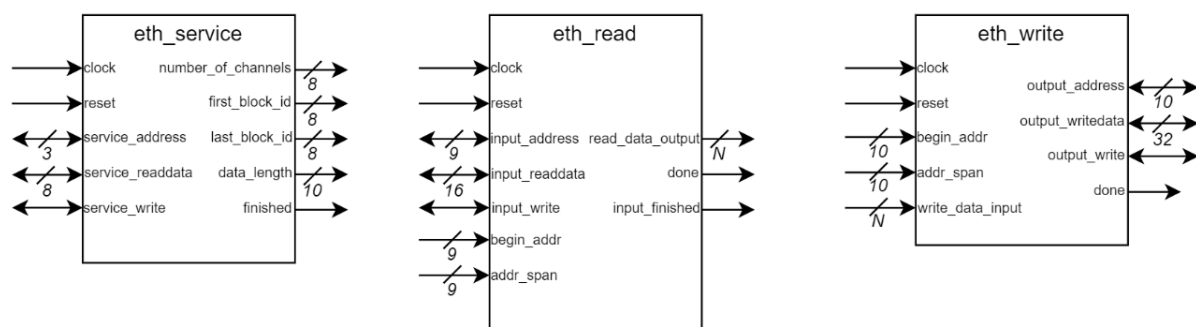


Рисунок 2.5 – Подмодули взаимодействия с памятью на чипе

Каждый модуль представляет из себя конечный автомат, выполняющий операции с блоком памяти на чипе. Рассмотрим каждый подмодуль подробнее.

2.2.3.1 Подмодуль чтения служебной информации

Листинг 2.1 содержит заголовок модуля *eth_service*.

Листинг 2.1 – Объявление и порты подмодуля *eth_service*

```
module eth_service (clock, reset, service_address,
service_readdata, service_write, number_of_channels,
first_block_id, last_block_id, data_length, finished);
```

Данный подмодуль производит выгрузку служебной информации из служебного блока памяти на чипе и запись ее в регистры в FPGA-части для дальнейшего использования, до начала работы устанавливает выходной сигнал *finished*, начинает работу по нижнему уровню входного сигнала *reset*, в конце работы сбрасывает сигнал *finished*. Взаимодействие с блоком памяти на чипе производится через двунаправленные порты *service_address* (шина адреса блока памяти), *service_readdata* (шина считывания данных с блока памяти), *service_write* (флаг записи данных в блок памяти, требует отдельного

выставления после каждой смены адреса). Подмодуль тактируется входным сигналом *clock* и имеет выходами считанные значения служебных признаков (*number_of_channels*, *first_block_id*, *last_block_id*, *data_length*).

2.2.3.2 Подмодуль чтения из входного блока памяти

Заголовок модуля *eth_read* представлен в листинге 2.2.

Листинг 2.2 – Объявление и порты подмодуля *eth_read*

```
module eth_read (clock, reset, input_address, input_readdata,  
input_write, begin_addr, addr_span, read_data_output, done,  
input_finished);  
parameter read_data_output_length = 8;
```

Данный подмодуль производит выгрузку данных из одного из блоков входных данных. Как и подмодуль *eth_service*, *eth_read* имеет три двунаправленных порта для взаимодействия с блоком памяти – *input_address*(шина адреса), *input_readdata*(шина считывания данных), *input_write*(флаг записи данных). Подмодуль начинает работу по нижнему уровню входного сигнала *reset* и до начала работы устанавливает выходные сигналы *input_finished* и *done*. Первый сигнал модуль сбрасывает по окончании считывания блока данных, начальный адрес *begin_addr* и размер *addr_span* которого задаются соответствующими входами, второй сигнал модуль импульсно сбрасывает после каждого считанного значения, считываемое значение подается на выходную шину *read_data_output*, ширина которой задается параметром *read_data_output_length*. Подмодуль тактируется сигналом *clock*.

2.2.3.3 Подмодуль записи полученных значений в блок выходных данных

В листинге 2.3 представлен заголовок модуля *eth_write*.

Листинг 2.3 – Объявление и порты модуля *eth_write*

```
module eth_write (clock, reset, output_address, output_writedata,
output_write, begin_addr, addr_span, write_data_input, done);
parameter write_data_input_length = 8;
```

Данный подмодуль производит загрузку значений на выходе тестируемого модуля (тестируемой цепочки?) в блок памяти на чипе для выходных данных. Как и предыдущие два подмодуля, *eth_write* взаимодействует с блоком памяти на чипе через три двунаправленных порта: шину адреса *output_address*, шину записи данных в блок *output_writedata* и флаг записи данных *output_write*, который выставляется для определенного адреса после произведенной операции записи. Область записи данных в блок памяти задается входами *begin_addr*(начальный адрес) и *addr_span*(количество данных для записи). Подмодуль до начала работы выставляет выходной сигнал *done*, начинает работу по нижнему уровню сигнала *reset*, по окончании работы сбрасывает сигнал *done*. Данные для записи в память подаются на шину *write_data_input*, ширина которой определяется параметром *write_data_input_length*. Подмодуль тактируется сигналом *clock*.

Для работы с несколькими блоками данных, как входных, так и выходных, можно объявлять несколько подмодулей *eth_read* или *eth_write*. В начале тестирования первым выполняется модуль *eth_service*, после окончания его работы в зависимости от условий тестирования запускаются модули *eth_read* и *eth_write*. Тестирование считается завершенным, когда завершает работу модуль *eth_write*.

2.3 Моделирование FPGA-подмодулей

Для проверки правильности работы аппаратных модулей была проведена симуляция в среде ModelSim. В качестве блоков памяти выступают пары вида «адрес - массив регистров», где адрес выступает индексом массива. Рассмотрим два примера, которые были описаны выше:

1. Комбинационный элемент, в данном случае возьмем 8-разрядный инвертор. Это простой пример, на котором можно промоделировать базовую

функциональность и в дальнейшем экспериментально проверить связь между внешним устройством и отладочной платой.

2. Модуль, имеющий внутреннюю память, в данном случае – пороговое устройство. Этот пример уже имеет непосредственное отношение к проекту радара и позволит промоделировать и проверить модуль в реальных условиях.

2.3.1 Пример 1 – 8-разрядный инвертор

Схематическое изображение соединения инвертора и подмодулей модуля сопряжение представлено на рисунке 2.6.

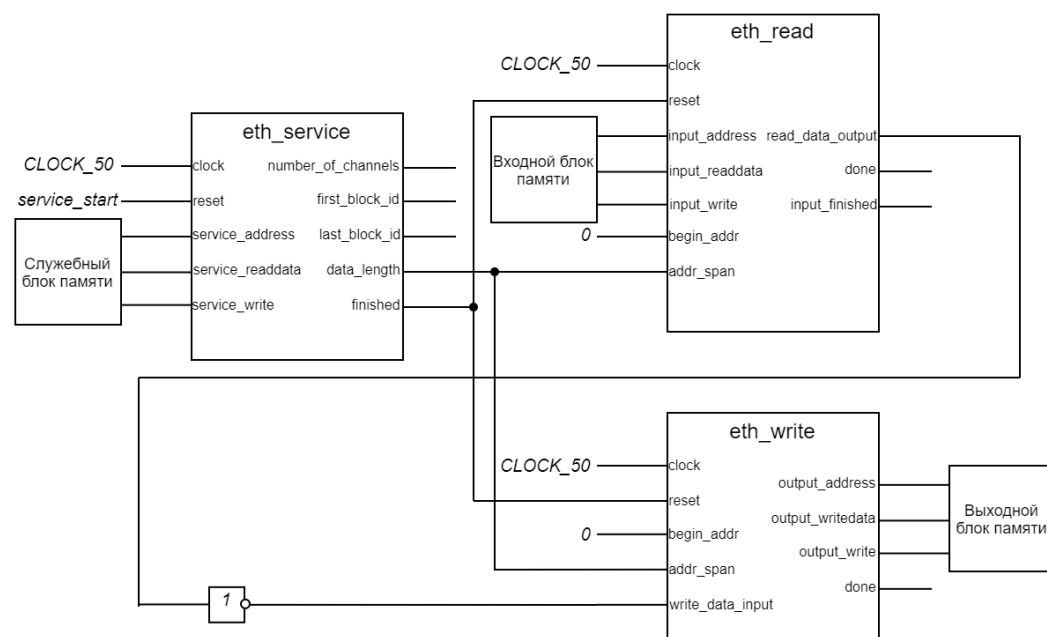


Рисунок 2.6 – Схема соединения модуля сопряжения и инвертора

В качестве данных для проверки была взята последовательность из 10 8-разрядных шестнадцатеричных чисел:

[0xFE, 0x54, 0x12, 0x32, 0x8A, 0xBA, 0x8C, 0xDE, 0x92, 0x11];

На выходе модуля *eth_write* в итоге ожидается последовательность:

[0x01, 0xAB, 0xED, 0xCD, 0x75, 0x45, 0x73, 0x21, 0x6D, 0xEE]

Результат моделирования представлен на рисунке 2.7.

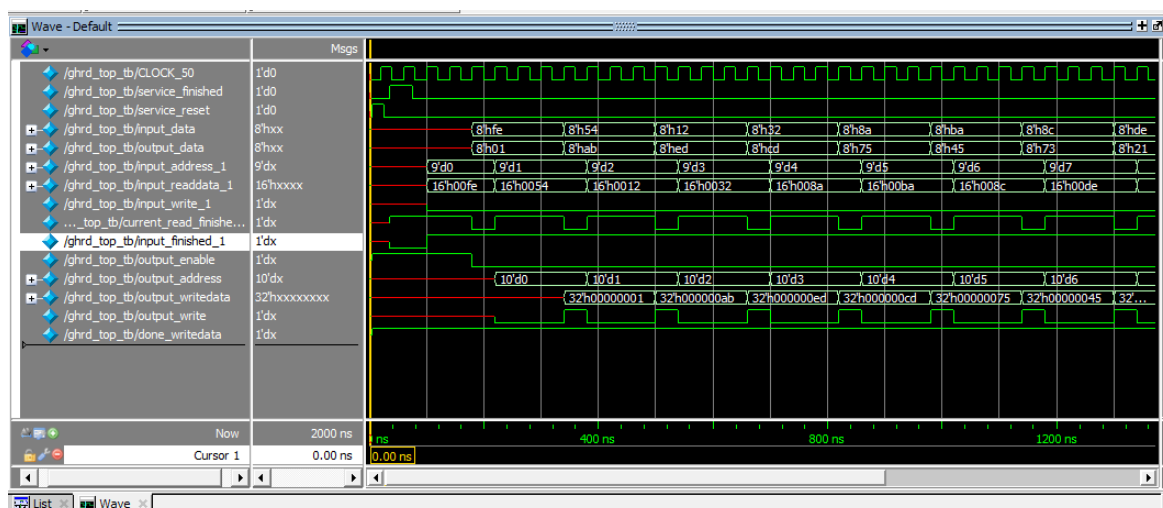
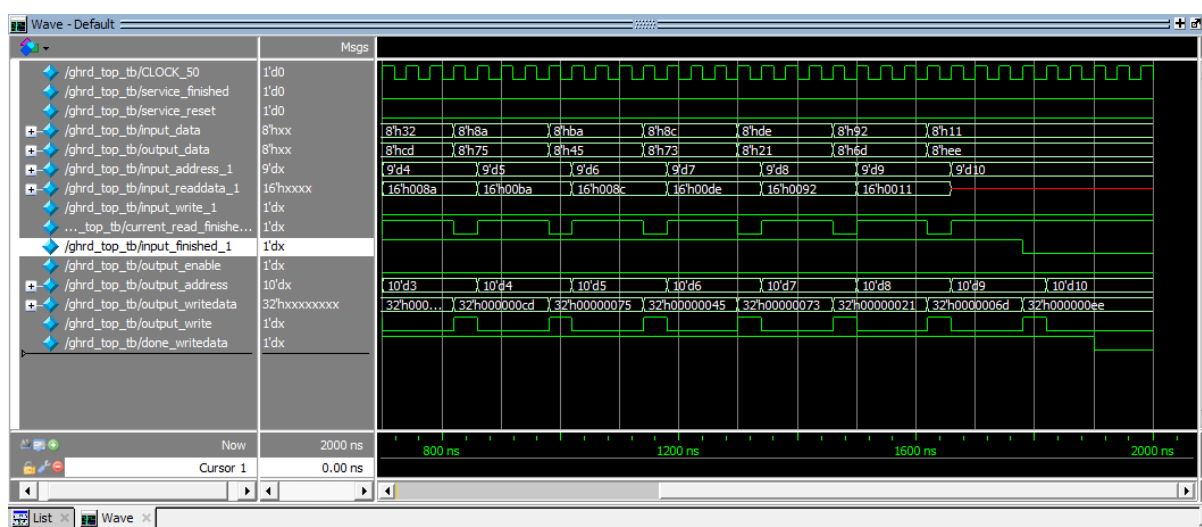


Рисунок 2.7 – Результат моделирования для примера 1



Продолжение рисунка 2.7

Сигнал *output_enable* подается на вход *reset* модуля *eth_write* и формируется с появлением первых данных на входе инвертора (шина *input_data*), выход инвертора – шина *output_data*. Шина *input_data* является выходом *read_data_output* модуля *eth_read*, а шина *output_data* – входом *write_data_input* модуля *eth_write*. В результате моделирования видно, что данные проходят через модули верно, на выходе *output_writedata* формируются верные значения.

2.3.2 Пример 2 – Пороговое устройство

Схематическое изображение порогового устройства представлено на рисунке 2.8.

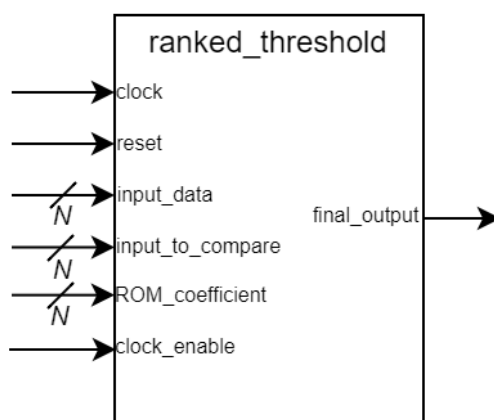


Рисунок 2.8 – Пороговое устройство

На вход порогового устройства подается массив N-разрядных данных (шина *input_data*), постоянный коэффициент из ПЗУ (*ROM_coefficient*), а также данные для сравнения (*input_to_compare*). Один из параметров модуля, *rank*, является рангом для выбора значения из *input_data*. Остальные параметры модуля для данной задачи рассматривать излишне.

Схематическое изображение соединения модуля порогового устройства и подмодулей модуля сопряжения представлено на рисунке 2.9.

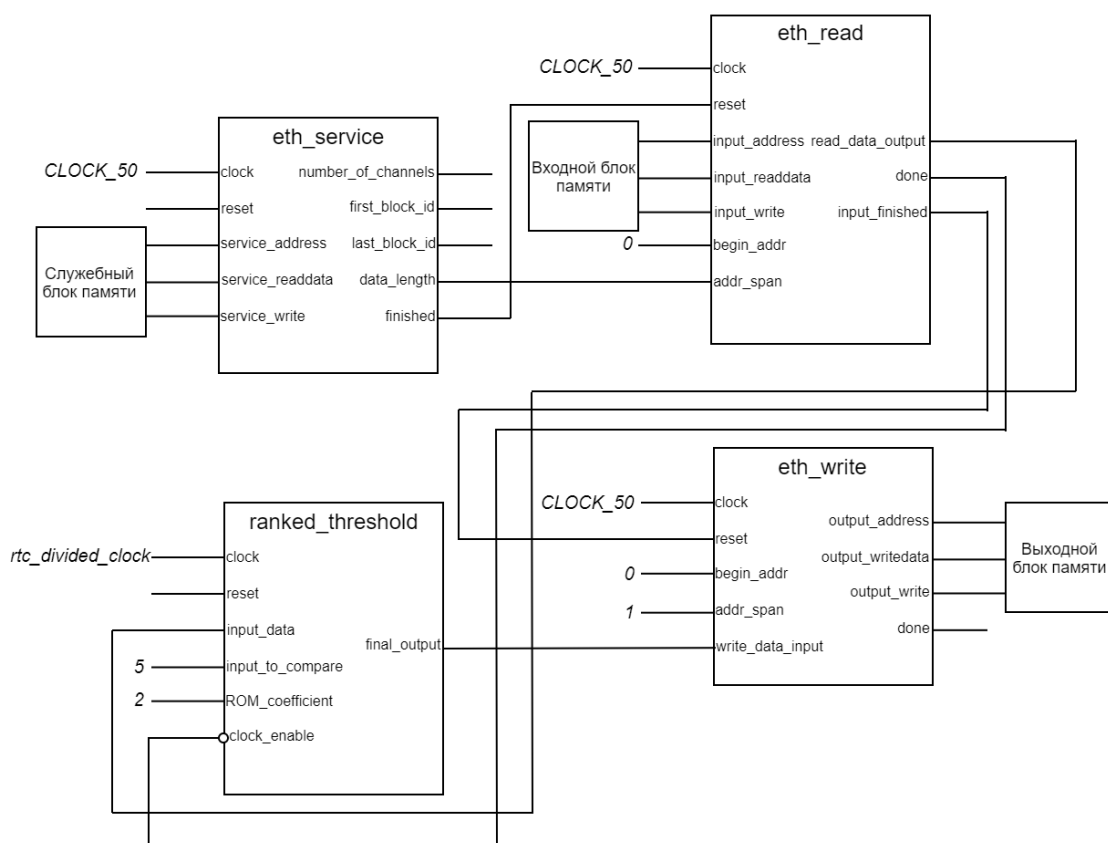


Рисунок 2.9 – Схема взаимного соединения модулей

Пороговое устройство тактируется сигналом *rtc_divided_clock* с частотой, равной половине главной частоты *CLOCK_50*.

Для моделирования тестирования порогового устройства выбраны следующие входные данные:

а) *input_data*: [1, 4, 9, 2, 6, 3, 7, 8, 5], подается с имитации блока памяти через блок *eth_read*

б) *ROM_coefficient*: 2

в) *input_to_compare*: 5

г) параметр *rank*: 1

При таких входных данных на выходе *final_output* ожидается значение 1.

Результат моделирования представлен на рисунке 2.10.

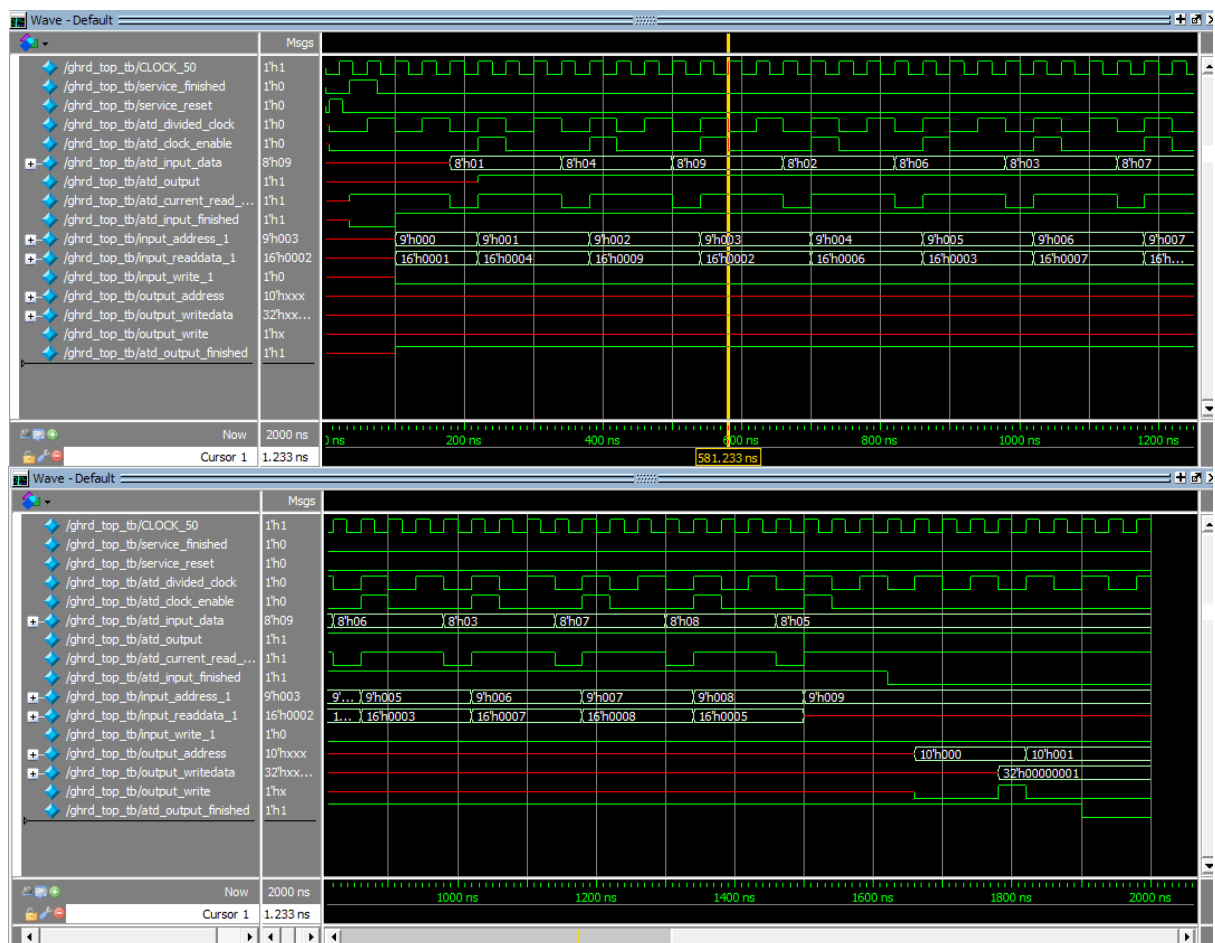


Рисунок 2.10 – Результат моделирования для примера 2.

Из результатов моделирования видно, что *eth_read* и *eth_write* отработали верно, первый передал нужные данные в пороговое устройство, *eth_write* считал правильное значение выхода устройства и записал его по нужному адресу.

2.4 Синтез FPGA-модулей

Модули для FPGA успешно синтезировались с помощью Quartus Prime Lite. Результаты синтеза в форме фрагментов RTL-диаграмм представлены на рисунках 2.11 – 2.13.

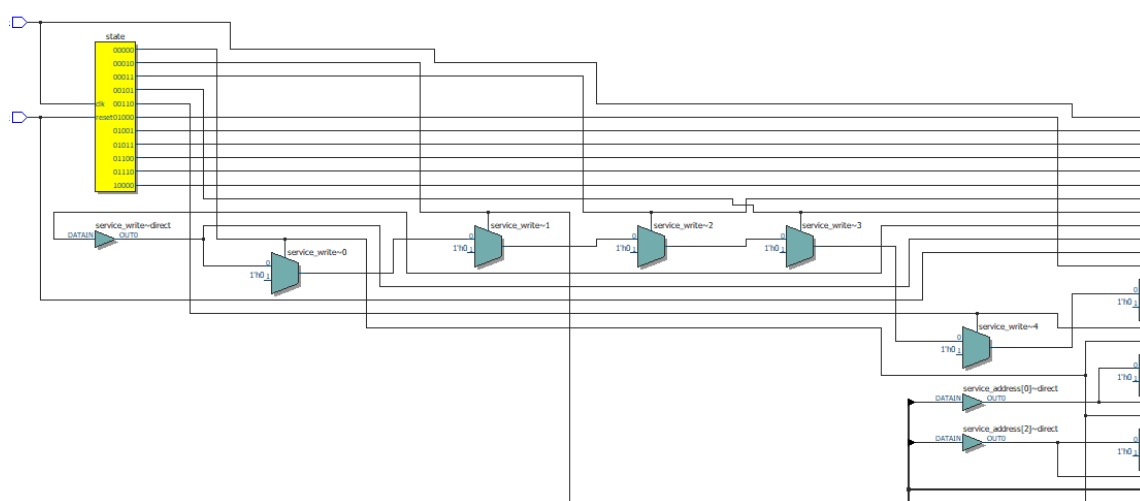


Рисунок 2.11 – Фрагмент RTL-диаграммы модуля *eth_service*.

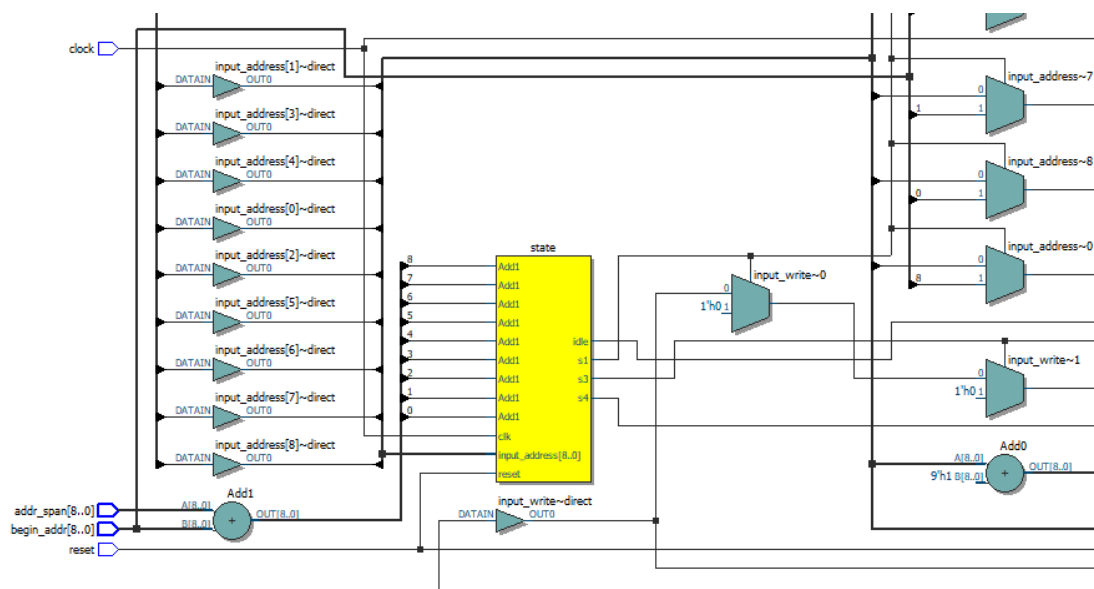
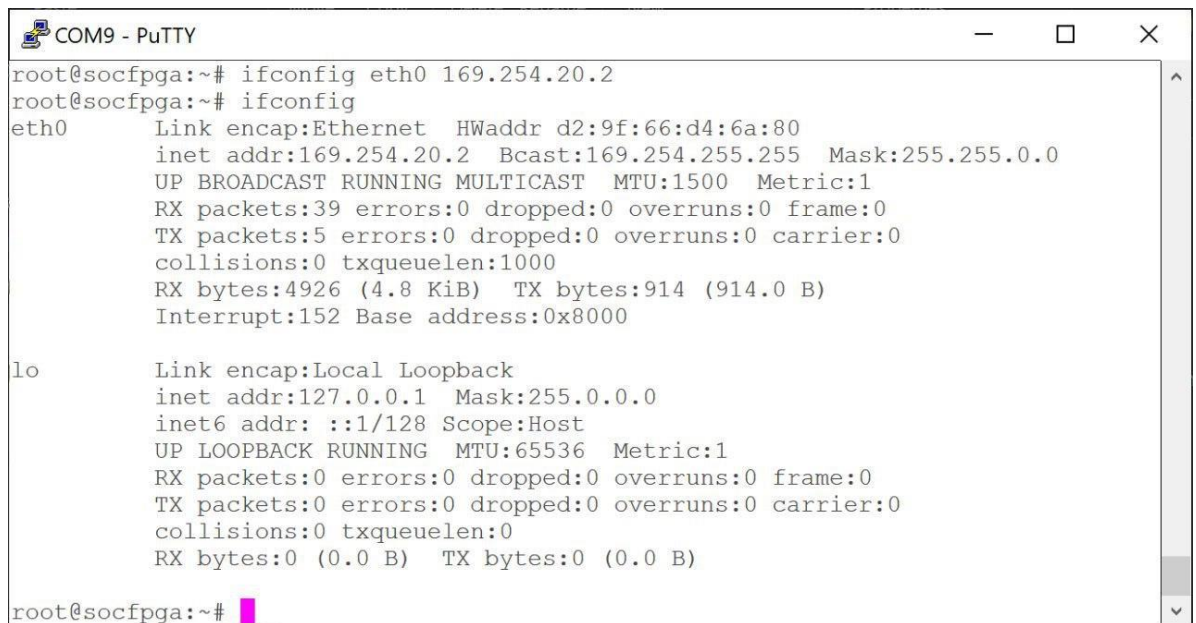


Рисунок 2.12 – Фрагмент RTL-диаграммы модуля *eth_read*.



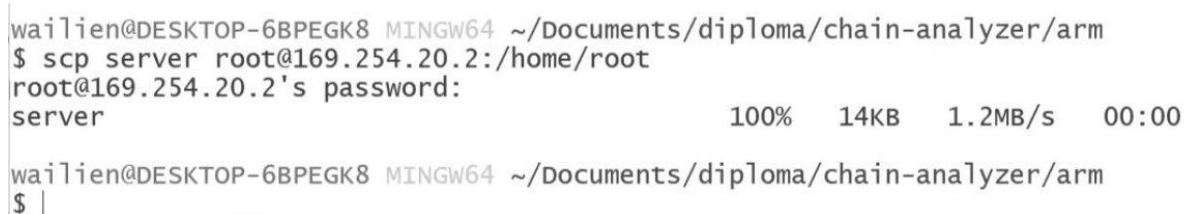
```
root@socfpga:~# ifconfig eth0 169.254.20.2
root@socfpga:~# ifconfig
eth0      Link encap:Ethernet  HWaddr d2:9f:66:d4:6a:80
          inet addr:169.254.20.2  Bcast:169.254.255.255  Mask:255.255.0.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:39 errors:0 dropped:0 overruns:0 frame:0
          TX packets:5 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:4926 (4.8 KiB)  TX bytes:914 (914.0 B)
          Interrupt:152 Base address:0x8000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

root@socfpga:~#
```

Рисунок 2.15 – Настройка IP-адреса платы

3. С помощью утилиты `scp` скомпилированный исполняемый файл сервера передается в плату. (рисунок 2.16)

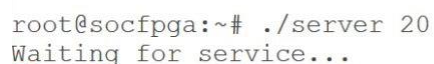


```
wailien@DESKTOP-6BPEGK8 MINGW64 ~/Documents/diploma/chain-analyzer/arm
$ scp server root@169.254.20.2:/home/root
root@169.254.20.2's password:
server                               100% 14KB  1.2MB/s   00:00

wailien@DESKTOP-6BPEGK8 MINGW64 ~/Documents/diploma/chain-analyzer/arm
$ |
```

Рисунок 2.16 – Передача файла сервера в плату

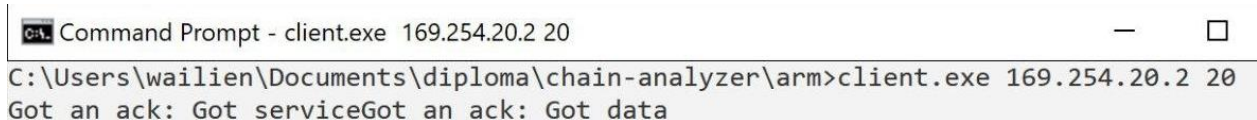
4. Запускается сервер, сервер начинает ожидать служебную датаграмму. (рисунок 2.17)



```
root@socfpga:~# ./server 20
Waiting for service...
```

Рисунок 2.17 – Запущенный сервер

5. На стороне PC запускается простой клиент, который отправляет служебную датаграмму и датаграмму данных серверу на ARM. (рисунок 2.18)



```
Command Prompt - client.exe 169.254.20.2 20
C:\Users\wailien\Documents\diploma\chain-analyzer\arm>client.exe 169.254.20.2 20
Got an ack: Got service
Got an ack: Got data
```

Рисунок 2.18 – Запуск клиента, отправка датаграмм

6. Сервер принял датаграммы, можно запускать обработку на FPGA.

(рисунок 2.19)

```
root@socfpga:~# ./server 20
Waiting for service...
Received a service datagram:
0001 0002 0003 0000 0009
Waiting for data...
Received a data datagram:
00010000 00040000 00090000 00020000 00060000 00030000 00070000 00080000 00050000

Press enter

dataLength = 9 00000009
Press enter to look at ocm
```

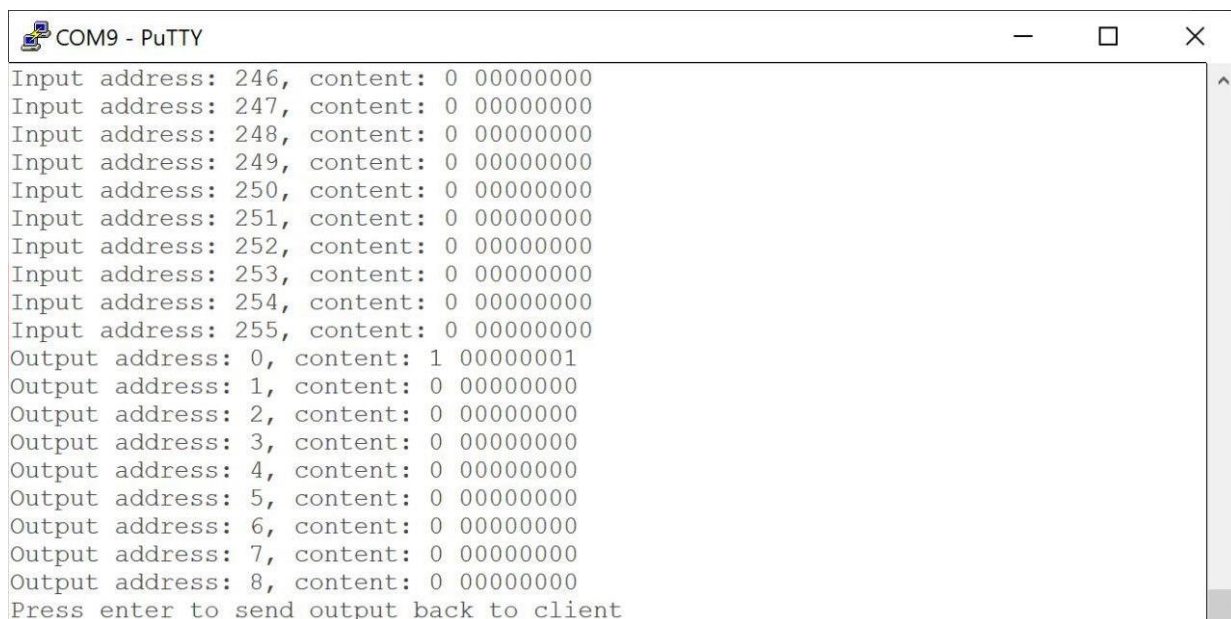
Рисунок 2.19 – Принятые сервером датаграммы

7. Обработка на FPGA завершена, появились выходные данные в соответствующем блоке памяти, смотрим память на сервере. (рисунок 2.20)

 COM9 - PuTTY

```
dataLength = 9 00000009
Press enter to look at ocm

Service address: 0, content: 1 00000001
Service address: 1, content: 2 00000002
Service address: 2, content: 3 00000003
Service address: 3, content: 0 00000000
Service address: 4, content: 9 00000009
Input address: 0, content: 1 00000001
Input address: 1, content: 4 00000004
Input address: 2, content: 9 00000009
Input address: 3, content: 2 00000002
Input address: 4, content: 6 00000006
Input address: 5, content: 3 00000003
Input address: 6, content: 7 00000007
Input address: 7, content: 8 00000008
Input address: 8, content: 5 00000005
Input address: 9, content: 0 00000000
Input address: 10, content: 0 00000000
Input address: 11, content: 0 00000000
Input address: 12, content: 0 00000000
```

```
COM9 - PuTTY
Input address: 246, content: 0 00000000
Input address: 247, content: 0 00000000
Input address: 248, content: 0 00000000
Input address: 249, content: 0 00000000
Input address: 250, content: 0 00000000
Input address: 251, content: 0 00000000
Input address: 252, content: 0 00000000
Input address: 253, content: 0 00000000
Input address: 254, content: 0 00000000
Input address: 255, content: 0 00000000
Output address: 0, content: 1 00000001
Output address: 1, content: 0 00000000
Output address: 2, content: 0 00000000
Output address: 3, content: 0 00000000
Output address: 4, content: 0 00000000
Output address: 5, content: 0 00000000
Output address: 6, content: 0 00000000
Output address: 7, content: 0 00000000
Output address: 8, content: 0 00000000
Press enter to send output back to client
```

Рисунок 2.20 – Состояние блоков памяти

8. Сервер увидел выходные данные, формируем и отправляем датаграмму с результатами клиенту. (рисунок 2.21)

```
Input address: 251, content: 0 00000000
Input address: 252, content: 0 00000000
Input address: 253, content: 0 00000000
Input address: 254, content: 0 00000000
Input address: 255, content: 0 00000000
Output address: 0, content: 1 00000001
Output address: 1, content: 0 00000000
Output address: 2, content: 0 00000000
Output address: 3, content: 0 00000000
Output address: 4, content: 0 00000000
Output address: 5, content: 0 00000000
Output address: 6, content: 0 00000000
Output address: 7, content: 0 00000000
Output address: 8, content: 0 00000000
Press enter to send output back to client

Got an ack: Got output. Press enter
```

Рисунок 2.21 – Отправка датаграммы с результатами клиенту

9. Клиент получил датаграмму с выходными данными, работа модуля успешно завершена. (рисунок 2.22)

```
C:\Users\wailien\Documents\diploma\chain-analyzer\arm>client.exe 169.254.20.2 20
Got an ack: Got serviceGot an ack: Got dataGot output:
1
C:\Users\wailien\Documents\diploma\chain-analyzer\arm>
```

Рисунок 2.22 – Выходные данные на клиенте

2.5.2 Результаты эксперимента

В результате проведения эксперимента протестирована работа одного из модулей тракта первичной обработки – порогового устройства. Обе составные части модуля (сервер на ARM и модули на FPGA) отработали успешно, клиент успешно получил результаты тестирования. В качестве клиентского приложения может выступать любое приложение, формирующее UDP-датаграммы нужного формата.

3 РАЗРАБОТКА МОДУЛЯ СОПРЯЖЕНИЯ С ИНТЕРФЕЙСОМ LINK

Данная глава посвящена разработке модуля сопряжения эмулятора комплексной системы радара с интерфейсом Link.

3.1 Описание функционирования модуля сопряжения с Link

Модуль сопряжения с Link построен на двух шаблонах – передатчике и приемнике. Приемник и передатчик работают в 4-битном режиме, передавая данные на обоих фронтах тактового сигнала. Данные представлены в виде блоков кадров слов, слова в кадрах слов 32-разрядные, всего кадр слов содержит 128 бит информации. Для того, чтобы принять, обработать и передать информацию обратно в DSP, на стороне FPGA требуется создать устройства обработки информации с приемника и передачи обработанной информации в передатчик.

Алгоритм работы устройства обработки информации с приемника:

1. Обнаружить появление первого слова кадра слов на приемнике.
2. Записать каждое слово кадра слов в буфер, из которого данные отправляются в обработку.
3. В случае обнаружения последнего слова последнего кадра слов блока выставить признак окончания приема.
4. Ожидать новый пакет кадров слов.

Алгоритм работы устройства для передачи обработанной информации передатчику:

1. Если передатчик готов принять данные для передачи, передать обработанные слова кадра слов в передатчик.
2. В случае неготовности передатчика выполнять дальнейшую передачу данных приостановить передачу очередного кадра слов и ожидать готовности передатчика.
3. В случае обнаружения последнего обработанного слова закончить передачу и перейти в режим ожидания новых обработанных данных.

Рассмотрим пример простой обработки данных – инверсию каждого принятого слова квадрослова и дальнейшую передачу инвертированных значений в DSP. В этом случае обработка принятых данных происходит практически мгновенно, поэтому после приема и обработки каждого квадрослова обработанные данные передаются в передатчик.

3.2 Описание программы

В программе использованы модули приемника и передатчика, а также разработаны устройства обработки данных со стороны FPGA. Схематическое изображение модулей приемника и передатчика представлено на рисунке 3.1.

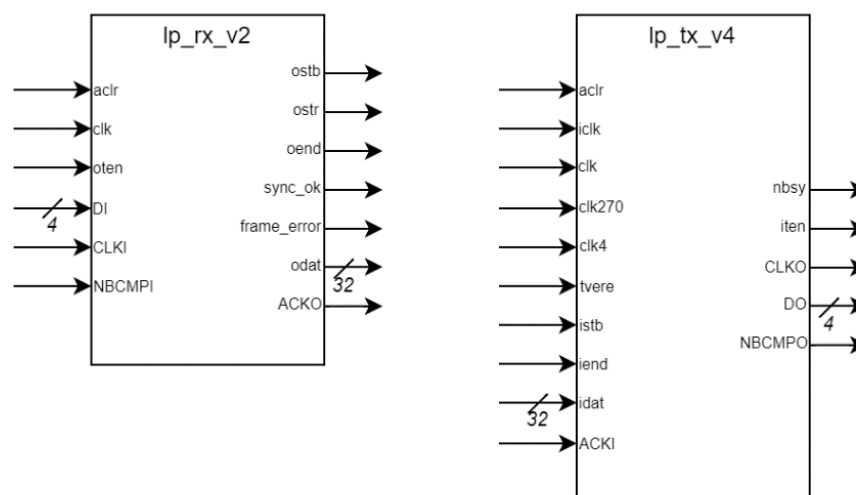


Рисунок 3.1 – Модули приемника и передатчика Link-порта

3.2.1 Модуль приемника Link-порта

Заголовок модуля приемника Link-порта представлен в листинге 3.1.

Листинг 3.1 – Объявление и порты модуля приемника

```
module lp_rx_v2(clk, aclr, DI, CLKI, ACKO, NBCMPI,
oten, ostb, ostr, oend, odat, sync_ok, frame_error);
parameter decimation_coeff = 2;
parameter ack_timeout = 20;
```

Модуль параметризуется двумя значениями: *decimation_coeff* (отношение входной и выходной частот домена, возможные значения: 1 и 2) и *ack_timeout* (таймаут приема).

DI, CLKI, ACKO и *NBCMPI* - интерфейсные сигналы Link-порта.

Модуль также имеет локальные сигналы для использования в FPGA, первый из которых – входной сигнал разрешения приема *oten*, на верхнем уровне которого устанавливается сигнал *ACKO* Link-порта, при подаче нижнего уровня сигнал *ACKO* сбрасывается в промежутке от 1-го до 3-го слова в квадрослове, при этом прием квадрослова доводится до конца. Также модуль имеет выходы *ostb*(признак принятия очередного слова данных), *ostr*(признак первого слова), *oend* (признак последнего слова последнего квадрослова), позволяющие отслеживать процесс приема данных. Принятые с порта данные подаются на выходную 32-разрядную шину *odat*. Выходной сигнал *sync_ok* является признаком получения полного пакета, от первого слова до сигнала *NBCMPI* с таймаутом после, после приема такого пакета выставляется верхний уровень сигнала. Сигнал *frame_error* выставляется, если обнаружен таймаут внутри квадрослова. Приемник тактируется сигналом *clk*, который выставляется равным частоте Link-порта, а также имеет вход асинхронного сброса *aclr*.

3.2.2 Модуль передатчика Link-порта

Заголовок модуля передатчика Link-порта представлен в листинге 3.2.

Листинг 3.2 – Объявление и порты модуля передатчика

```
module lp_tx_v4 (aclr, iclk, clk, clk270, clk4, tvere, nbsy,
  idat, istb, iend, iten, CLKO, DO, ACKI, NBCMPO);
parameter    DEVICE = "Stratix II";
parameter    FIFO_DEPTH = 32;
parameter    FIFO_AFULL_THR = 24;
```

Модуль имеет три параметра: *DEVICE*(имя устройства, в которое будет загружаться модуль), *FIFO_DEPTH*(объем внутреннего FIFO-буфера) и *FIFO_AFULL_THR*(порог для формирования сигнала о том, что буфер практически заполнен).

DO, CLKO, ACKI и *NBCMPO* - интерфейсные сигналы Link-порта.

Как и приемник, передатчик имеет локальные сигналы. Входной сигнал *tvere* – признак необходимости проверки передаваемых данных, выходной сигнал *nbsy* – признак наличия данных во входном FIFO-буфере, выходной сигнал *iten* – признак готовности к передаче. Передаваемые данные подаются на 32-разрядную входную шину *idat* и сопровождаются сигналом *istb*, после передачи последнего слова вырабатывается сигнал *iend*. Модуль имеет сигнал асинхронного сброса *aclr* и тактируется четырьмя тактовыми сигналами: *iclk* (частота входных данных), *clk* (частота Link Port), *clk270* (*iclk*, сдвинутый по фазе на 270°) и *clk4* (*iclk*, разделенный на 4). Для формирования нужных частот используется модуль PLL, поставляемый со средством разработки Quartus компании Intel.

3.2.3 Устройства обработки данных

Для осуществления задачи приема, обработки, в данном случае инвертирования, и передачи, были разработаны поведенческие описания конечных автоматов для обработки данных с приемника и передачи результатов обработки в передатчик. Схематические изображения автоматов представлены на рисунке 3.2.

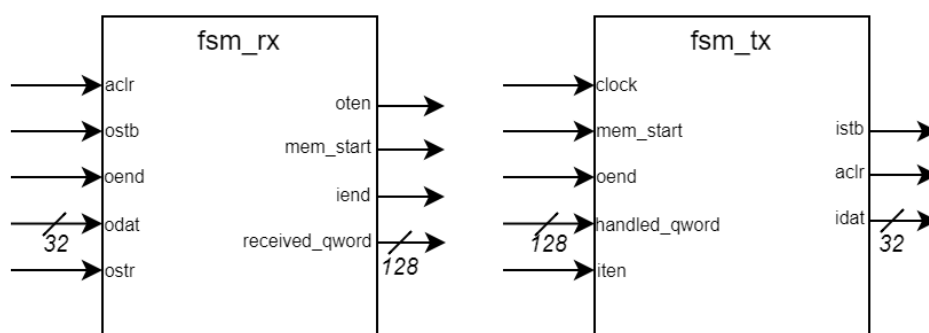


Рисунок 3.2 – Автоматы работы с данными на стороне FPGA.

Здесь *fsm_rx* – автомат обработки данных с приемника, *fsm_tx* – автомат передачи результатов обработки данных в передатчик.

3.2.3.1 Автомат обработки данных с приемника

Автомат имеет собственный сигнал сброса *aclr*, по нулевому уровню которого начинает работу, тактируется сигналом приемника *ostb* и чувствителен к сигналу *oend*. В начале нового цикла работы автомат

выставляет сигнал *oten*, далее подает сигнал сброса на автомат передачи результатов обработки и при наличии верхнего уровня сигнала *ostr* записывает первое слово квадрослова в буфер, после чего по сигналу *ostb* записывает в буфер три следующих слова квадрослова. Данные из буфера затем передаются на обработку. После записи в буфер последнего слова квадрослова автомат сбрасывает сигнал *oten* и выставляет нижний уровень сигнала сброса автомата передатчика, в результате чего тот начинает работу, после чего переходит в режим ожидания. Если автомат обнаруживает верхний фронт сигнала *oend*, он выставляет сигнал *iend*, тем самым уведомляя передатчик об окончании потока данных.

3.2.3.2 Автомат передачи результатов обработки в передатчик

Автомат имеет сигнал *mem_start*, по нижнему уровню которого автомат начинает работу, также выполняющий функцию сигнала сброса. Тактирование автомата происходит по сигналу *clock*, соответствующий частоте порта связи, умноженной на 2. Это сделано для обеспечения правильности передачи данных в приемник. Автомат также чувствителен к сигналу *oend*. По верхнему уровню сигнала *mem_start* автомат выставляет нижний уровень сигнала *istb*, инициализирует нулевыми значениями шину *idat*, а также сбрасывает сигнал *oend_found*, являющийся признаком появления сигнала *oend*. Далее, при условии готовности передатчика (верхний уровень сигнала *iten*) автомат передает на шину *idat* первое обработанное слово квадрослова *handled_qword*, а также выставляет сигнал *istb*, если передатчик не готов, тогда автомат сбрасывает сигнал *istb* и ожидает верхнего уровня *iten*. Если после обработки первого слова квадрослова автомат обнаруживает верхний уровень сигнала *oend*, то выставляется признак *oend_found*, далее по тому же принципу, что и для первого слова, на шину *idat* передаются оставшиеся три слова квадрослова. После отправки в передатчик последнего слова квадрослова автомат вырабатывает импульс сигнала *aclr*, что сбрасывает в начальное состояние автомат обработки данных приемника и начинает новый цикл его работы,

после чего переходит в режим ожидания. Если в процессе работы автомата был выставлен признак *oend_found*, то автомат с задержкой на один такт сбрасывает сигнал *istb* и переходит в режим ожидания нового сигнала сброса.

3.3 Моделирование процесса приема, обработки и передачи

Для симуляции взаимодействия DSP и FPGA было произведено моделирование устройства в ModelSim. Для имитации взаимодействия с DSP в testbench были объявлены две пары приемник-передатчик, одна имитирует приемник и передатчик на стороне DSP, другая пара приемник-передатчик управляется с FPGA. В качестве обработки использовалась инверсия. Схема соединения двух пар, а также обозначения соединяющих их сигналов, показаны на рисунке 3.3.

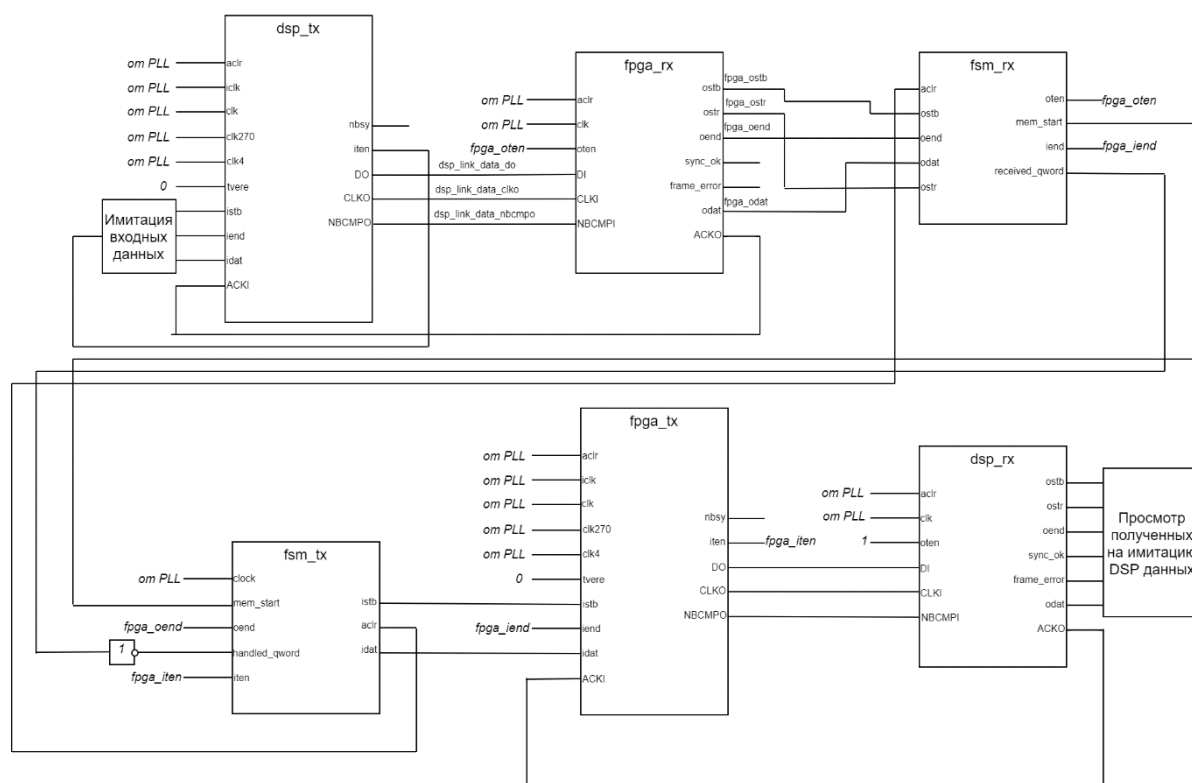


Рисунок 3.3 – Схема соединения пар приемник-передатчик

На рисунке подписи над соединительными линиями обозначают названия сигналов в testbench, названия сигналов слева или справа от линий — названия входных и выходных сигналов соответственно.

Для эмуляции отправления данных со стороны DSP в testbench разработан автомат, считывающий входные данные из файла. Тестовая

загружаться, поскольку используется дифференциальная передача данных, требующая корректного назначения каждой дифференциальной пары. В данном случае модуль сопряжения загружается в FPGA Cyclone 10 LP компании Intel. Таблица с назначением контактов модуля приведена в приложении Д.

Описание модуля сопряжения успешно синтезировалось средствами системы Quartus Prime Lite. Результат синтеза в виде фрагмента RTL-диаграммы представлен на рисунке 3.9. Полная RTL-диаграмма модуля сопряжения приведена в приложении Е.

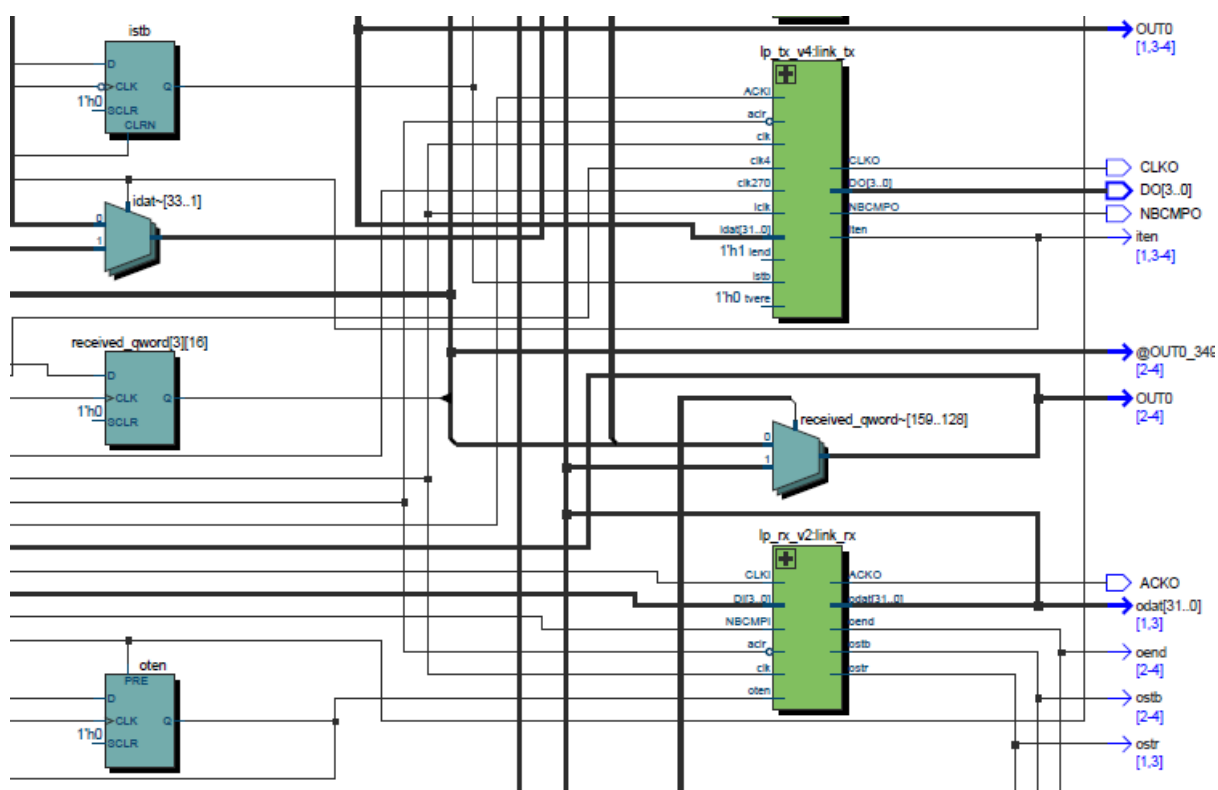


Рисунок 3.9 Фрагмент RTL-диаграммы модуля сопряжения с Link.

3.5 Выводы по главе

В результате был разработан модуль сопряжения с интерфейсом Link, основанный на шаблонных модулях приемника и передатчика, успешно произведено моделирование взаимодействия имитированного DSP и FPGA, успешно произведен синтез модуля.

4 ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ МОДУЛЕЙ СОПРЯЖЕНИЯ

В данной главе приведено технико-экономическое обоснование разработки модулей сопряжения эмулятора радара. Был произведен анализ протоколов и способов реализации модулей сопряжения, разработка и тестирование модулей, для одного из модулей была произведена экспериментальная проверка на ПЛИС с использованием макетной платы. Разработка носит практический характер, модули сопряжения являются частью комплексной системы обработки сигналов с радара.

В рамках проведения технико-экономического обоснования бакалаврской работы были выполнены следующие действия:

- составлен план-график выполнения работ;
- произведена оценка величины заработной платы исполнителей с учетом взносов на обязательное страхование;
- рассчитаны затраты на приобретение материалов и специального оборудования;
- произведен подсчет амортизационных отчислений;
- рассчитана сметная стоимость разработки;

4.1 Составление плана-графика выполнения работ

Первым этапом при расчете полных затрат на выполнение разработки является составление детализированного плана-графика работ, которые необходимо произвести в ходе разработки устройства. План-график работ является основой для расчета затрат на этапе проектирования, для выполнения расчета также нужно определить исполнителей работ и трудоемкость каждой работы. В данном обосновании единица измерения трудоемкости работ – человеко-дни. План-график с указанием исполнителей и трудоемкости работ представлен в таблице 4.1.

Таблица 4.1 План-график выполнения работ

№ работы	Наименование работы	Исполнитель	Длительность работы, человеко-дни
1	Разработка и выдача технического задания	Руководитель	2
2	Получение и изучение задания	Студент	2
3	Обзор источников по теме работы	Студент	7
4	Анализ протоколов интерфейсов и способов реализации модулей сопряжения	Студент	10
5	Выбор способа реализации для каждого модуля сопряжения	Студент	2
6	Разработка модулей сопряжения	Студент	16
7	Моделирование и тестирование модулей сопряжения	Студент	8
8	Экспериментальная проверка модуля сопряжения с Ethernet	Студент	3
9	Консультации с научным руководителем	Руководитель	7
		Студент	7
10	Оформление пояснительной записки	Студент	10

Таким образом, на выполнение всех работ было затрачено:

- студентом: 65 ч. дн.
- руководителем: 9 ч. дн.

4.2 Расчет затрат на оплату труда исполнителей

Одной из основных статей расходов на разработку устройства является заработная плата занятых в разработке исполнителей с учетом обязательных отчислений.

4.2.1 Расчет основной заработной платы исполнителей

На основе данных о месячных заработных платах руководителя и студента (принимается равной заработной плате инженера) был произведен расчет оплаты одного человеко-дня каждого исполнителя. Месячный оклад руководителя составляет 47500 рублей, студента – 28000 рублей. Стоимость человеко-дня вычисляется путем деления месячного оклада исполнителя на количество рабочих дней в месяце, в данной работе принято равным 21.

Тогда стоимость человеко-дня составляет:

– для руководителя: $СЧД_{рук} = \frac{47500}{21} = 2\,261,9$ руб/день;

– для студента: $СЧД_{студ} = \frac{28000}{21} = 1\,333,33$ руб/день;

Используя полученные значения стоимости человеко-дня можно рассчитать основную заработную плату исполнителей, рассчитываемую путем умножения затраченных на работу человеко-дней на стоимость человеко-дня. Тогда основная заработная плата составляет:

– для руководителя: $З_{осн.з/пл\ рук} = 2\,261,9 \cdot 9 = 20\,357,1$ руб.;

– для студента: $З_{осн.з/пл} = 1\,333,33 \cdot 65 = 86\,666,67$ руб.;

4.2.2 Расчет дополнительной заработной платы

После расчета основной заработной платы производится расчет дополнительной заработной платы по формуле:

$$З_{доп.з/пл} = З_{осн.з/пл} \cdot \frac{H_{доп}}{100}, \quad (4.1)$$

где $З_{доп.з/пл}$ – дополнительная заработная плата исполнителей (руб.), $З_{осн.з/пл}$ – основная заработная плата исполнителей (руб.), $H_{доп}$ – норматив дополнительной заработной платы, в данной работе принятый за 14%.

В соответствии с формулой 4.1 и данными из пункта 4.2.1 дополнительная заработная плата составит:

– для руководителя: $З_{доп.з/пл\ рук} = 20\,357,1 \cdot \frac{14}{100} = 2\,849,99$ руб.;

– для студента: $З_{доп.з/пл\ студ} = 86\,666,67 \cdot \frac{14}{100} = 12\,133,33$ руб.;

4.2.3 Расчет обязательных социальных отчислений

Отчисления в сторону страховых взносов на обязательное страхование (медицинское, социальное и пенсионной) вычисляется с основной и дополнительной заработной платы по формуле:

$$З_{соц} = (З_{осн.з/пл} + З_{доп.з/пл}) \cdot \frac{H_{соц}}{100}, \quad (4.2)$$

где $З_{соц}$ – отчисления на социальные нужды с заработной платы (руб.), $H_{соц}$ – норматив отчислений на страховые взносы на обязательное страхование,

равный 30%, $Z_{\text{доп.з./пл}}$ – дополнительная заработная плата исполнителей (руб.), $Z_{\text{осн.з./пл}}$ – основная заработная плата исполнителей (руб.).

В соответствии с формулой 4.2 и данными из пунктов 4.2.1 и 4.2.2 отчисления на социальные нужды составляют:

– для руководителя: $Z_{\text{соц рук.}} = (20\,357,1 + 2\,849,99) \cdot \frac{30}{100} = 6\,962,13$ руб.;

– для студента: $Z_{\text{соц студ.}} = (86\,666,67 + 12\,133,33) \cdot \frac{30}{100} = 29\,640$ руб.;

4.2.4 Итоговые затраты на оплату труда

Итоги по расходам на оплату труда представлены в таблице 4.2.

Таблица 4.2 – Расходы на оплату труда исполнителей

Исполнитель	Основная заработная плата, руб.	Дополнительная заработная плата, руб.	Обязательные страховые отчисления, руб.	Итого, руб
Руководитель	20 357,1	2 849,99	6 962,13	30 169,22
Студент	86 666,67	12 133,33	29 640	128 440
Общие затраты на оплату труда, руб.				158 609,22

Таким образом, общие расходы по статье «Оплата труда основных исполнителей» составили 158 609,22 рублей.

4.3 Расчет затрат на приобретение материалов и спецоборудования

Расчет затрат на сырье и материалы производится по формуле:

$$Z_m = \sum_{i=1}^L G_i C_i \left(1 + \frac{H_{\text{т.з.}}}{100}\right), \quad (4.3)$$

где Z_m – затраты на сырье и материалы (руб.), i – индекс вида сырья или материала, G_i – норма расхода i -го материала на единицу продукции (ед.), C_i – цена приобретения единицы i -го материала (руб. за единицу), $H_{\text{т.з.}}$ – норма транспортных затрат, в данной работе принята за 10%.

Перечень расходных материалов и затрат на их приобретение представлен в таблице 4.3. Для проведения эксперимента с модулем сопряжения с Ethernet необходимо приобретение макетной платы. Эксперимент был проведен с использованием макетной платы DE1-SoC

фирмы Terasic, стоимость которой без НДС (ставка НДС на момент выполнения ВКР – 20%) составляет 25960 рублей.

Таблица 4.3 – Затраты на материалы.

Изделие	Тип	Норма расхода изделия, ед.	Цена за единицу изделия, руб./ед.	Транспортные расходы, руб.	Затраты на изделие, руб.
Бумага офисная	A4, 500 листов	1	306	30,6	336,60
Картридж для принтера	Лазерный, черный	1	3601	360,1	3961,10

4.4 Расчет амортизационных отчислений

Амортизационные отчисления по i -му основному средству рассчитываются по формуле:

$$A_i = C_{п.н.i} \cdot \frac{H_{ai}}{100}, \quad (4.4)$$

где A_i – амортизационные отчисления за год по i -му основному средству (руб.); $C_{п.н.i}$ – первоначальная стоимость i -го основного средства (руб.); H_{ai} – годовая норма амортизации i -го основного средства (%); Норма амортизации рассчитывается как:

$$H_{ai} = \frac{1}{T_{п.и.}} \cdot 100\%, \quad (4.5)$$

где $T_{п.и.}$ – срок полезного использования основного средства.

Амортизационные отчисления по i -му основному средству, используемому при работе над ВКР, определяются как:

$$A_{iВКР} = A_i \cdot \frac{T_{iВКР}}{12}, \quad (4.6)$$

где $A_{iВКР}$ – амортизационные отчисления по i -му основному средству, используемому при работе над ВКР (руб.); A_i – амортизационные отчисления по i -му основному средству за год (руб.); $T_{iВКР}$ – время, в течение которого используется i -е основное средство (мес.).

Смета по расчету амортизационных отчислений по основным средствам, используемым в выпускной работе, приведена в таблицах 4.4 и 4.5.

Таблица 4.4 – Амортизационные отчисления за год

Основное средство	Первоначальная стоимость, руб.	Срок полезного использования, лет	Годовая норма амортизации, %	Амортизационные отчисления за год, руб
Ноутбук Lenovo ThinkPad X230	28000	3	33%	9333,33
Отладочная плата Terasic DE1-SoC	25960	3	33%	8653,33
МФУ Xerox B205	11945	3	33%	3981,66

Таблица 4.5 Амортизационные отчисления за период выполнения ВКР

Основное средство	Амортизационные отчисления за год, руб	Время выполнения ВКР, месяцев	Амортизационные отчисления, руб
Ноутбук Lenovo ThinkPad X230	9333,33	3	2333,33
Отладочная плата Terasic DE1-SoC	8563,33	3	2140,83
МФУ Xerox B205	3981,66	3	995,42

Таким образом, общие затраты на амортизационные отчисления составляют $2\,333,33 + 2\,140,83 + 995,42 = 5\,469,58$ рублей.

4.5 Расчет накладных расходов

В рамках данной работы сумма накладных расходов рассчитывается как доля в 20% от общей стоимости разработки. Тогда накладные расходы на разработку составляют:

$$Z_{\text{накл}} = (158609,22 + 4297,7 + 5469,58 + 25960) \cdot \frac{20}{100} = 38\,867,3 \text{ рублей}$$

4.6 Расчет сметной стоимости разработки

Смета по статьям расходов, а также расчет структуры себестоимости представлен в таблице 4.6.

Таблица 4.6 Структура полных затрат на разработку модулей сопряжения

Наименование статьи	Сумма, руб.	Доля в общей стоимости, %
Расходы на оплату труда	122 007,09	52,32
Отчисления на социальные нужды	36 602,13	15,695
Материалы	4 297,7	1,84
Амортизационные отчисления	5 469,58	2,345
Накладные расходы	38 867,3	16,67
Спецоборудование	25 960	11,13
Итого	233 203,8	100

4.7 Выводы по главе

В данной главе был произведен расчет полных затрат на разработку модулей сопряжения эмулятора радара.

Полные затраты на разработку составляют 233 тысячи 203 рубля и 80 копеек, основные статьи расходов: оплата труда 52,32%; накладные расходы 16,67%; отчисления на социальные нужды 15,695%.

ЗАКЛЮЧЕНИЕ

В результате работы разработаны модули сопряжения с Ethernet (отладочный модуль) и Link (модуль для связи с DSP) для эмулятора комплексной системы обнаружения и обработки сигналов радара, которые позволяют наладить связь между первичным трактом обработки и средой тестирования или же внешним процессором обработки сигналов. С помощью отладочного интерфейса на реальной плате был оттестирован один из модулей первичной обработки, что подтверждает способность отладочного модуля сопряжения работать в реальных условиях. Модуль для связи с DSP был отмоделирован с имитацией DSP, что позволяет с высокой степенью уверенности полагать, что модуль отработает и в реальных условиях. Таким образом, модули сопряжения уже сейчас полезны как часть разрабатываемой системы.

Развитием данной работы может быть оптимизация модулей сопряжения по времени и занимаемым ресурсам, проверка модуля сопряжения с Link на реальной плате с реальным DSP.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГИБДД МВД РФ. Сведения о показателях безопасности дорожного движения. – <http://stat.gibdd.ru/>
2. Всемирная организация здравоохранения. 10 причин смерти в мире. – <https://www.who.int/ru/news-room/fact-sheets/detail/the-top-10-causes-of-death>
3. Singh, S. (2015, February). Critical reasons for crashes investigated in the National Motor Vehicle Crash Causation Survey. (Traffic Safety Facts Crash Stats. Report No. DOT HS 812 115). Washington, DC: National Highway Traffic Safety Administration.
4. Создание отечественного высокотехнологичного производства систем безопасности автотранспорта на основе блока управления и интеллектуальных датчиков, включающих миллиметровые радары диапазона 76-77 ГГц: отчет о НИР (заключ.): Санкт-Петербургский государственный электротехнический университет «ЛЭТИ» им. В.И.Ульянова (Ленина); рук. Шичкина Ю. А.; исполн.: Куприянов М. С. [и др.]. – С-Пб, 2019. – 778 с. – № НИОКТР АААА-А19-119121890135-3
5. Cisco Certified Network Associate. Кадр Ethernet: введение. – <http://ccna.mpei.ac.ru/IntroductionToNetworkTech/course/module5/5.1.2.3/5.1.2.3.html>
6. IEEE Standards Association. Standard Group MAC Addresses: A Tutorial Guide. – <https://standards.ieee.org/content/dam/ieee-standards/standards/web/documents/tutorials/macgrp.pdf>
7. Intel Corporation. Triple-Speed Ethernet IP User Guide. – https://www.intel.com/content/dam/www/programmable/us/en/pdfs/literature/ug/ug_ethernet.pdf
8. Материалы CCNA на русском. Протокол TCP – назначение и функционал. – <http://ciscotips.ru/tcp>
9. Материалы CCNA на русском. Протокол UDP. – <http://ciscotips.ru/udp>
10. Intel Corporation. Nios II Processors for FPGA. – <https://www.intel.com/content/www/us/en/products/programmable/processor/nios-ii.html>

11. Altera. Introduction to Cyclone V Hard Processor System. – https://people.ece.cornell.edu/land/courses/ece5760/DE1_SOC/HPS_INTRO_54001.pdf
12. Altera. Introduction to Altera Nios II Soft Processor. – https://people.ece.cornell.edu/land/courses/ece5760/DE2/tut_nios2_introduction.pdf
13. Carsten Pinkle. The Why and How of Differential Signaling. – <https://www.allaboutcircuits.com/technical-articles/the-why-and-how-of-differential-signaling/>
14. Texas Instruments. LVDS Application and Data Handbook. – <https://www.ti.com/lit/ug/slld009/slld009.pdf>
15. Texas Instruments. LVDS Owner's Manual. – <http://www.ti.com/lit/ug/snla187/snla187.pdf>
16. Загорский В. Сопряжение схем дифференциальной логики разных типов // Компоненты и технологии. 2004, вып. № 1. С. 56–58.
17. АО ПКК «Миландр». ТСКЯ.431281.002СП. Сигнальный процессор со статической суперскалярной архитектурой 1967BH028, K1967BH028, K1967BH028K, 1967BH02H4, K1967BH02H4.
18. Altera. Analog Devices Link-Port Reference Design. – <https://www.intel.com/content/dam/www/programmable/us/en/pdfs/literature/an/an332.pdf>
19. Altera. Design Guidelines for Implementing LVDS Interfaces in Cyclone Series Devices.– <https://www.intel.com/content/dam/www/programmable/us/en/pdfs/literature/an/an479.pdf>
20. Денисов А. О документировании проектов для ПЛИС // Компоненты и технологии. 2010, вып. № 9. С. 70–72.
21. ГОСТ 19.101–77. Единая система программной документации (ЕСПД). Виды программ и программных документов (с Изменением N 1). М.: Стандартинформ, 2010.
22. ГОСТ 19.103–77. ГОСТ 19.103-77 ЕСПД. Обозначения программ и программных документов. М.: Стандартинформ, 2010

ПРИЛОЖЕНИЕ А. Система HPS-SoC в Platform Designer

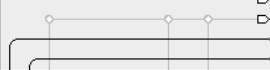
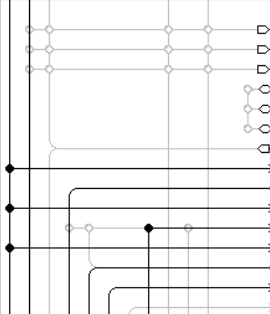
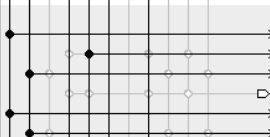
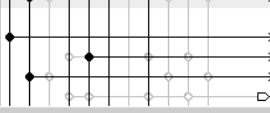
Use	Connections	Name	Description	Export	Clock	Base
☒		clk_0 clk_in clk_in_reset clk clk_reset	Clock Source Clock Input Reset Input Clock Output Reset Output	clk reset Double-click to export Double-click to export	exported	
☒		hps_0 f2h_cold_reset_req f2h_debug_reset_req f2h_warm_reset_req f2h_stm_hw_events memory hps_io h2f_reset h2f_axi_clock h2f_axi_master f2h_axi_clock f2h_axi_slave h2f_jw_axi_clock h2f_jw_axi_master f2h_irq0 f2h_irq1	Arria V/Cyclone V Hard Processor System Reset Input Reset Input Reset Input Conduit Conduit Conduit Reset Output Clock Input AXI Master Clock Input AXI Slave Clock Input AXI Master Interrupt Receiver Interrupt Receiver	hps_0_f2h_cold_reset_req hps_0_f2h_debug_reset_req hps_0_f2h_warm_reset_req hps_0_f2h_stm_hw_events memory hps_0_hps_io hps_0_h2f_reset Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export	clk_0	
☒		ocm_service clk1 s1 reset1 s2 clk2 reset2	On-Chip Memory (RAM or ROM) Clock Input Avalon Memory Mapped Slave Reset Input Avalon Memory Mapped Slave Clock Input Reset Input	Double-click to export Double-click to export Double-click to export ocm_service_export Double-click to export Double-click to export	clk_0	0x0000_0000
☒		ocm_data_input_re clk1 s1 reset1 s2	On-Chip Memory (RAM or ROM) Clock Input Avalon Memory Mapped Slave Reset Input Avalon Memory Mapped Slave	Double-click to export Double-click to export Double-click to export ocm_data_input_re_export	clk_0	0x0000_1000

Рисунок А.1 – Начало системы, HPS и первые два блока памяти

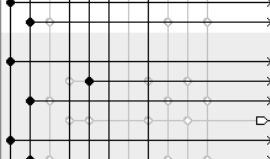
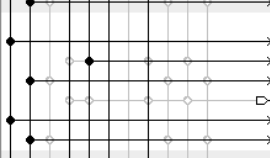
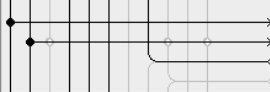
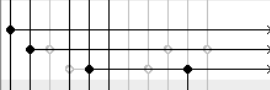
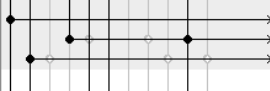
Use	Connections	Name	Description	Export	Clock	Base
☒		clk2 reset2 ocm_data_input_im clk1 s1 reset1 s2 clk2 reset2	Clock Input Reset Input On-Chip Memory (RAM or ROM) Clock Input Avalon Memory Mapped Slave Reset Input Avalon Memory Mapped Slave Clock Input Reset Input	Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export ocm_data_input_im_export Double-click to export Double-click to export	clk_0	0x0000_1200
☒		ocm_data_output_re clk1 s1 reset1 s2 clk2 reset2	On-Chip Memory (RAM or ROM) Clock Input Avalon Memory Mapped Slave Reset Input Avalon Memory Mapped Slave Clock Input Reset Input	Double-click to export Double-click to export Double-click to export Double-click to export ocm_data_output_re_export Double-click to export Double-click to export	clk_0	0x0000_2000
☒		hps_only_master clk clk_reset master master_reset	JTAG to Avalon Master Bridge Clock Input Reset Input Avalon Memory Mapped Master Reset Output	Double-click to export Double-click to export Double-click to export Double-click to export	clk_0	
☒		sysid_qsys clk reset control_slave	System ID Peripheral Clock Input Reset Input Avalon Memory Mapped Slave	Double-click to export Double-click to export Double-click to export	clk_0	0x0001_0000
☒		onchip_memory2_0 clk1 s1 reset1	On-Chip Memory (RAM or ROM) Clock Input Avalon Memory Mapped Slave Reset Input	Double-click to export Double-click to export Double-click to export	clk_0	0x0000_0000
☒		jtag_uart clk reset avalon_jtag_slave	JTAG UART Clock Input Reset Input Avalon Memory Mapped Slave	Double-click to export Double-click to export Double-click to export	clk_0	0x0002_0000

Рисунок А.2 – Остальные используемые блоки памяти и служебные блоки системы

Use	Connections	Name	Description	Export	Clock	Base
		clk_reset	Reset Input	<i>Double-click to export</i>		
		master	Avalon Memory Mapped Master	<i>Double-click to export</i>	[clk]	
		master_reset	Reset Output	<i>Double-click to export</i>		
☒		sysid_qsys	System ID Peripheral			
		clk	Clock Input	<i>Double-click to export</i>	clk_0	
		reset	Reset Input	<i>Double-click to export</i>	[clk]	
		control_slave	Avalon Memory Mapped Slave	<i>Double-click to export</i>	[clk]	0x0001_0000
☒		onchip_memory2_0	On-Chip Memory (RAM or ROM)			
		clk1	Clock Input	<i>Double-click to export</i>	clk_0	
		s1	Avalon Memory Mapped Slave	<i>Double-click to export</i>	[clk1]	0x0000_0000
		reset1	Reset Input	<i>Double-click to export</i>	[clk1]	
☒		jtag_uart	JTAG UART			
		clk	Clock Input	<i>Double-click to export</i>	clk_0	
		reset	Reset Input	<i>Double-click to export</i>	[clk]	
		avalon_jtag_slave	Avalon Memory Mapped Slave	<i>Double-click to export</i>	[clk]	0x0002_0000
		irq	Interrupt Sender	<i>Double-click to export</i>	[clk]	
☒		fpga_only_master	JTAG to Avalon Master Bridge			
		clk	Clock Input	<i>Double-click to export</i>	clk_0	
		clk_reset	Reset Input	<i>Double-click to export</i>	[clk]	
		master	Avalon Memory Mapped Master	<i>Double-click to export</i>		
		master_reset	Reset Output	<i>Double-click to export</i>		
☒		intr_capturer_0	Interrupt Capture Module			
		clock	Clock Input	<i>Double-click to export</i>	clk_0	
		reset_sink	Reset Input	<i>Double-click to export</i>	[clock]	
		avalon_slave_0	Avalon Memory Mapped Slave	<i>Double-click to export</i>	[clock]	0x0003_0000
		interrupt_receiver	Interrupt Receiver	<i>Double-click to export</i>	[clock]	
☒		ocm_data_output_im	On-Chip Memory (RAM or ROM)			
		clk1	Clock Input	<i>Double-click to export</i>	clk_0	
		s1	Avalon Memory Mapped Slave	<i>Double-click to export</i>	[clk1]	0x0000_2400
		reset1	Reset Input	<i>Double-click to export</i>	[clk1]	
		s2	Avalon Memory Mapped Slave	ocm_data_output_im_export	[clk2]	
		clk2	Clock Input	<i>Double-click to export</i>	clk_0	
		reset2	Reset Input	<i>Double-click to export</i>	[clk2]	

Рисунок А.3 – Конец системы

ПРИЛОЖЕНИЕ Б. Код программы ARM-сервера

```
/* Creates a datagram server. The port
   number is passed as an argument. This
   server runs forever */
#include <sys/types.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <string.h>
#include <netdb.h>
#include <stdio.h>
#include <sys/mman.h>
#include <fcntl.h>
#include "hwlib.h"
#include "socal/socal.h"
#include "socal/hps.h"
#include "socal/alt_gpio.h"
#include "hps_0.h"

volatile uint8_t * ocm_service = NULL ;
volatile uint16_t* ocm_input_re = NULL ;
volatile uint16_t* ocm_input_im = NULL ;
volatile uint32_t * ocm_output_re = NULL ;

#define LOW_ADDRESS 0x0000FFFF
#define HIGH_ADDRESS 0xFFFF0000

#define HW_REGS_BASE ( ALT_STM_OFST )
#define HW_REGS_SPAN ( 0x04000000 )
#define HW_REGS_MASK ( HW_REGS_SPAN - 1 )

void error(const char *msg) {
    perror(msg);
    exit(0);
}

int main(int argc, char *argv[]) {
    //for ocm mapping
    void *virtual_base;
    int fd;
    int addr;

    //for UDP
    int serviceSuccess, dataSuccess;
    int sock, length, n;
    socklen_t fromlen;
    struct sockaddr_in server;
    struct sockaddr_in from;
    uint8_t dgramService[5];
    uint32_t dgramData[256];
    uint32_t dgramOutput[512];
    uint16_t dataLength = 0x0;
    char ack_buffer[256];

    if (argc < 2) {
        fprintf(stderr, "error: no port provided\n");
        exit(0);
    }

    //opening dev/mem
```

```

if( ( fd = open( "/dev/mem", ( O_RDWR | O_SYNC ) ) ) == -1 ) {
    printf( "error: could not open \"/dev/mem\"...\n" );
    return( 1 );
}

//mapping
virtual_base = mmap( NULL, HW_REGS_SPAN, ( PROT_READ | PROT_WRITE ),
                    MAP_SHARED, fd, HW_REGS_BASE );
if( virtual_base == MAP_FAILED ) {
    printf( "error: mmap() failed...\n" );
    close( fd );
    return( 1 );
}

//initializing ocm base address
ocm_service = (uint8_t*)(virtual_base + ((unsigned long)(ALT_LWFPGASLVS_OFST +
    OCM_SERVICE_BASE) & (unsigned long)(HW_REGS_MASK)));
ocm_input_re = (uint16_t*)(virtual_base + (( unsigned long)(ALT_LWFPGASLVS_OFST
    + OCM_DATA_INPUT_RE_BASE) & (unsigned long)(HW_REGS_MASK)));
ocm_input_im = (uint16_t*)(virtual_base + ((unsigned long)(ALT_LWFPGASLVS_OFST +
    OCM_DATA_INPUT_IM_BASE) & (unsigned long)(HW_REGS_MASK)));
ocm_output_re = (uint32_t*)(virtual_base + (( unsigned long)(ALT_LWFPGASLVS_OFST
    + OCM_DATA_OUTPUT_RE_BASE) & (unsigned long)(HW_REGS_MASK)));

//opening socket, initializing server
sock = socket(AF_INET, SOCK_DGRAM, 0);
if (sock < 0)
    error("error: opening socket");
length = sizeof(server);
bzero(&server,length);
server.sin_family=AF_INET;
server.sin_addr.s_addr=INADDR_ANY;
server.sin_port=htons(atoi(argv[1]));
if (bind(sock,(struct sockaddr *)&server,length)<0)
    error("binding");
fromlen = sizeof(struct sockaddr_in);

//clearing the buffers
for (n = 0; n < 5; n++) dgramService[n] = 0x00;
for (n = 0; n < 256; n++) dgramData[n] = 0x00000000;

while (1) {
    //waiting for service datagram
    puts("Waiting for service...");

    //receiving service datagram
    serviceSuccess = recvfrom(sock, dgramService,5,0,
        (struct sockaddr *)&from, &fromlen);
    if (serviceSuccess < 0) error("recvfrom, service");

    puts("Received a service datagram: ");

    for (n = 0; n < 5; n++) printf("%04X ", dgramService[n]);
    putchar('\n');
    //letting client know we received the service datagram
    n = sendto(sock, "Got service", 11, 0, (struct sockaddr *)&from,fromlen);
    if (n < 0) error("sendto service"); /*
    //waiting for data datagram
    puts("Waiting for data...");

    //receiving data datagram
    dataSuccess = recvfrom(sock,dgramData,1024,0,

```

```

        (struct sockaddr *)&from,&fromlen);
if (dataSuccess < 0) error("recvfrom, data");
//printing received data
puts("Received a data datagram: ");
for (n = 0; n < 256; n++) if (dgramData[n] & 0xFFFFFFFF)
    printf("%08X ", dgramData[n]);
putchar('\n');
//letting client know we received the data datagram
n = sendto(sock, "Got data", 8, 0, (struct sockaddr *)&from,&fromlen);
if (n < 0) error("sendto data");
puts("Press enter");
getchar();
//getting data length
dataLength = (uint16_t)((dgramService[3] << 8) & 0xFF00) |
    (uint16_t)((dgramService[4]) & 0x00FF);
printf("dataLength = %d %08X\n", dataLength, dataLength);
//putting info into ocm
for (addr = 0; addr < 5; addr++)
    *(ocm_service+addr) = dgramService[addr];

    for (addr = 0, n = 0; addr < 256; addr++) {
        *(ocm_input_im+addr) = (uint16_t)(dgramData[n] & LOW_ADDRESS);
        *(ocm_input_re+addr) = (uint16_t)((dgramData[n] & HIGH_ADDRESS) >> 16);
        n++;
    }
puts("Press enter to look at ocm");
getchar();
//looking at ocm
for (addr = 0; addr < 5; addr++)
    printf("Service address: %d, content: %d %08X\n", addr,
        (*(ocm_service+addr)), (*(ocm_service+addr)));
for (addr = 0; addr < 256; addr++)
    printf("Input address: %d, content: %d %08X\n", addr,
        (*(ocm_input_re+addr)), (*(ocm_input_re+addr)));
for (addr = 0; addr < 256; addr++)
    printf("Input address: %d, content: %d %08X\n", addr,
        (*(ocm_input_im+addr)), (*(ocm_input_im+addr)));
for (addr = 0; addr < dataLength; addr++)
    printf("Output address: %d, content: %d %08X\n", addr,
        (*(ocm_output_re+addr)), (*(ocm_output_re+addr)));
//waiting for signal to form output datagram
puts("Press enter to send output back to client");
getchar();
//forming the output datagram
for (addr = 0; addr < dataLength; addr++)
{
    dgramOutput[addr] = (*(ocm_output+addr));
}
//sending the output datagram to client
n = sendto(sock,dgramOutput,dataLength*4,0,
    (struct sockaddr *)&from,&fromlen);
if (n < 0) error("sendto output");
//getting an ack from the client that it received the datagram
n = recvfrom(sock, ack_buffer,256,0,
    (struct sockaddr *)&from,&fromlen);
if (n < 0) error("recvfrom output");
write(1,"Got an ack: ",12);
write(1,ack_buffer,n);
puts("Press enter");
getchar();
}
return 0;
}

```


ПРИЛОЖЕНИЕ В. RTL-диаграммы подмодулей модулей сопряжения

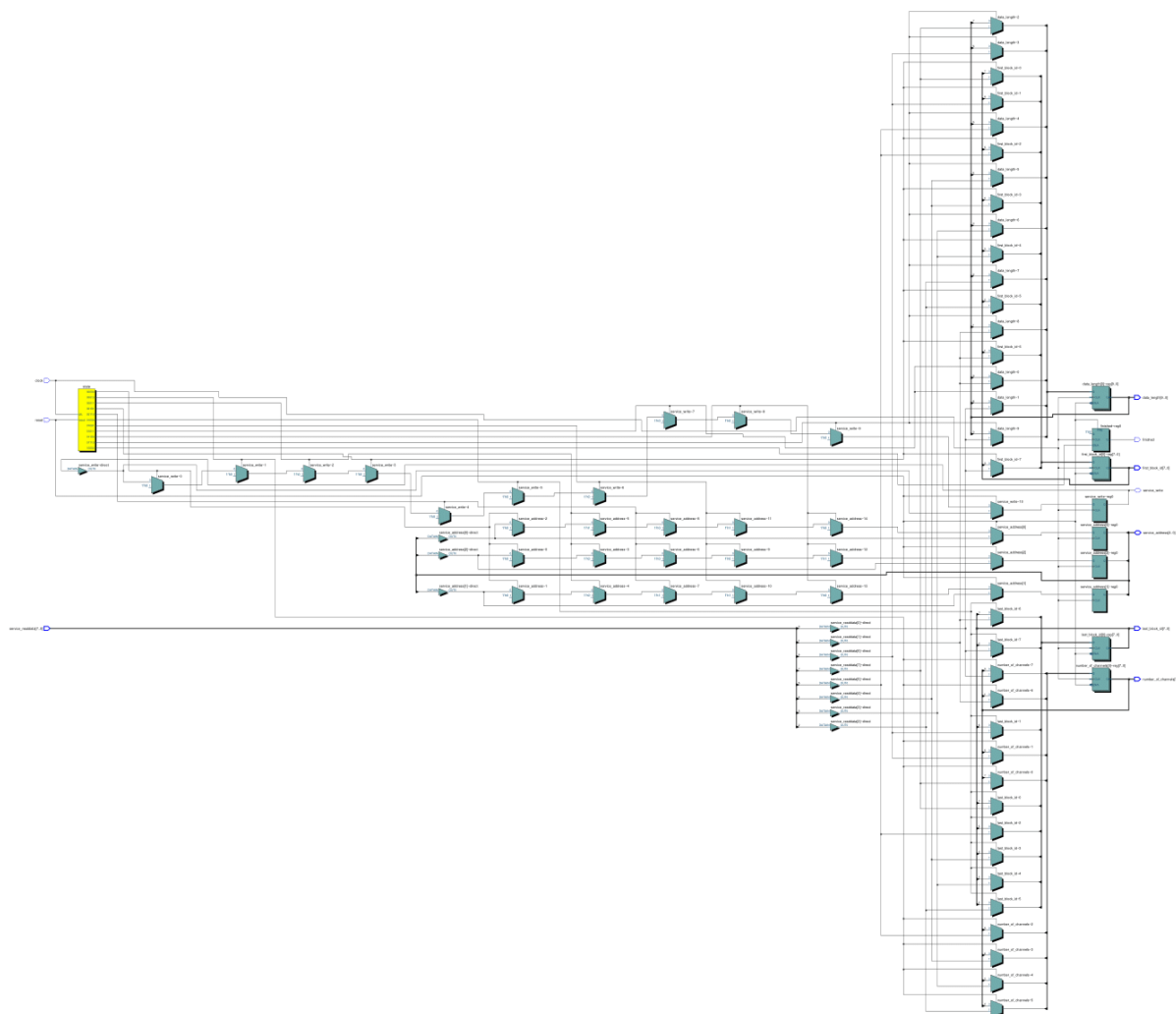


Рисунок В.1 – Подмодуль *eth_service*

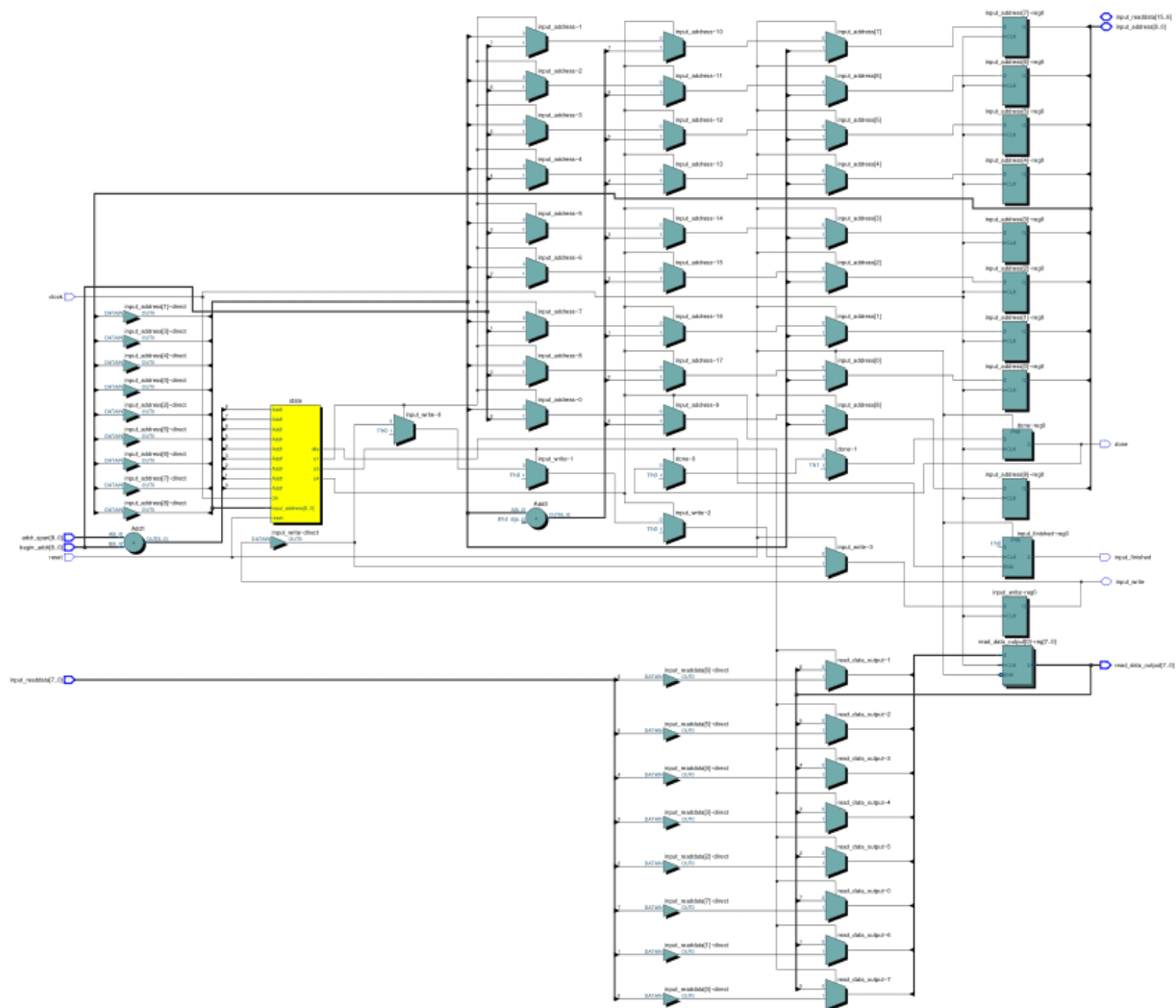


Рисунок В.2 – Подмодуль `eth_read`

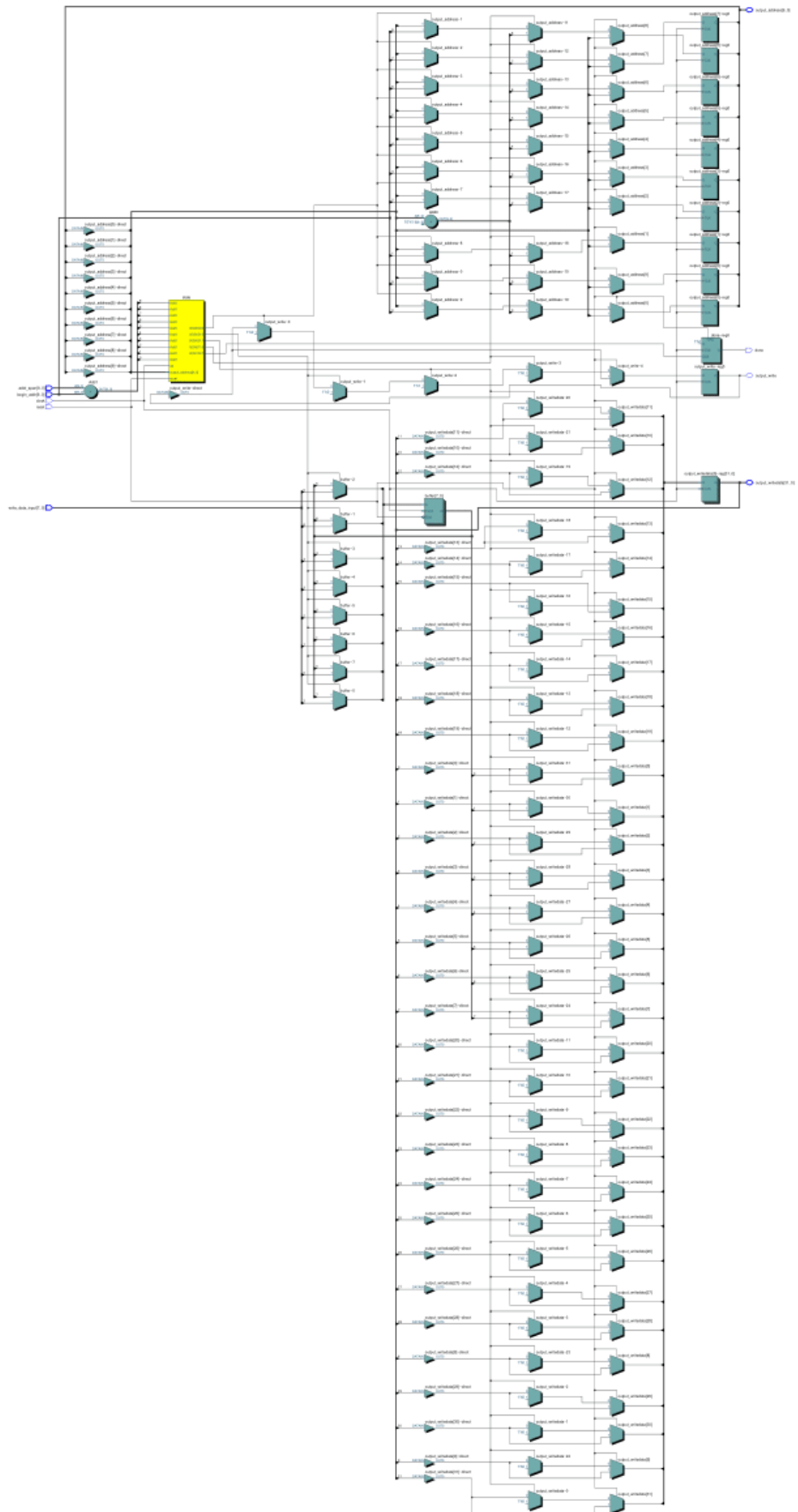


Рисунок В.3 – Подмодуль *eth_write*

ПРИЛОЖЕНИЕ Г. Подробные результаты моделирования модуля сопряжения с Link

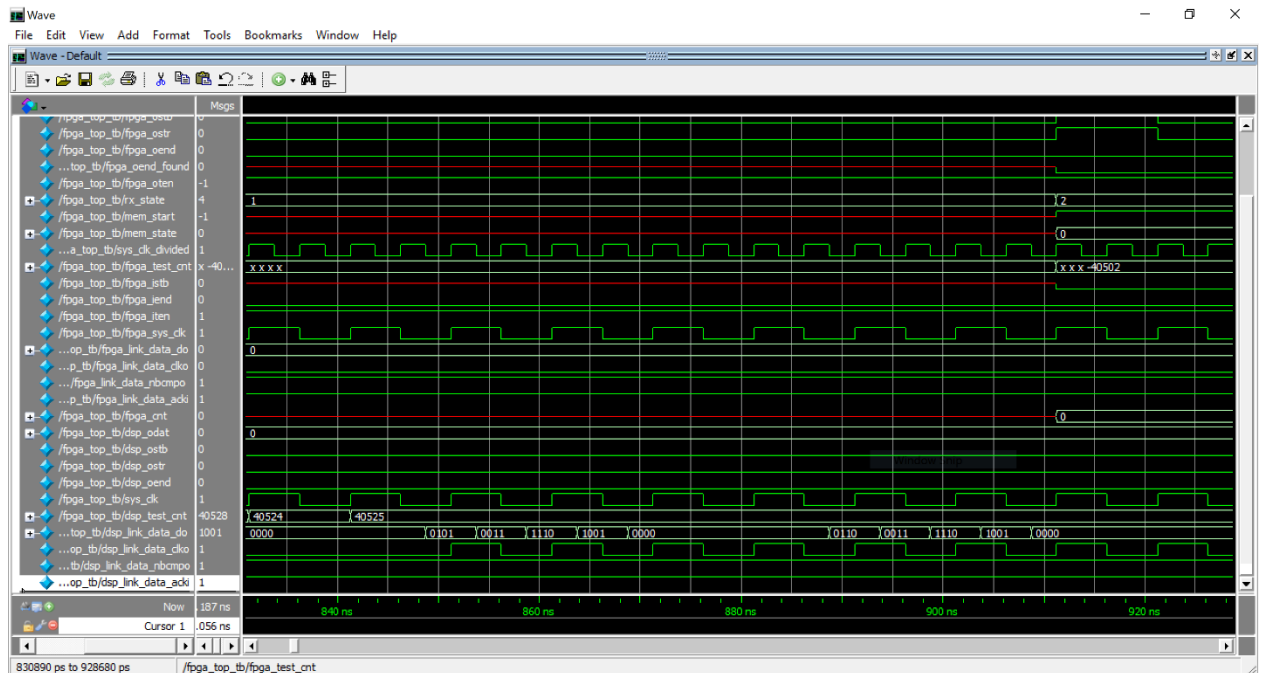


Рисунок Г.1 – Начало моделирования

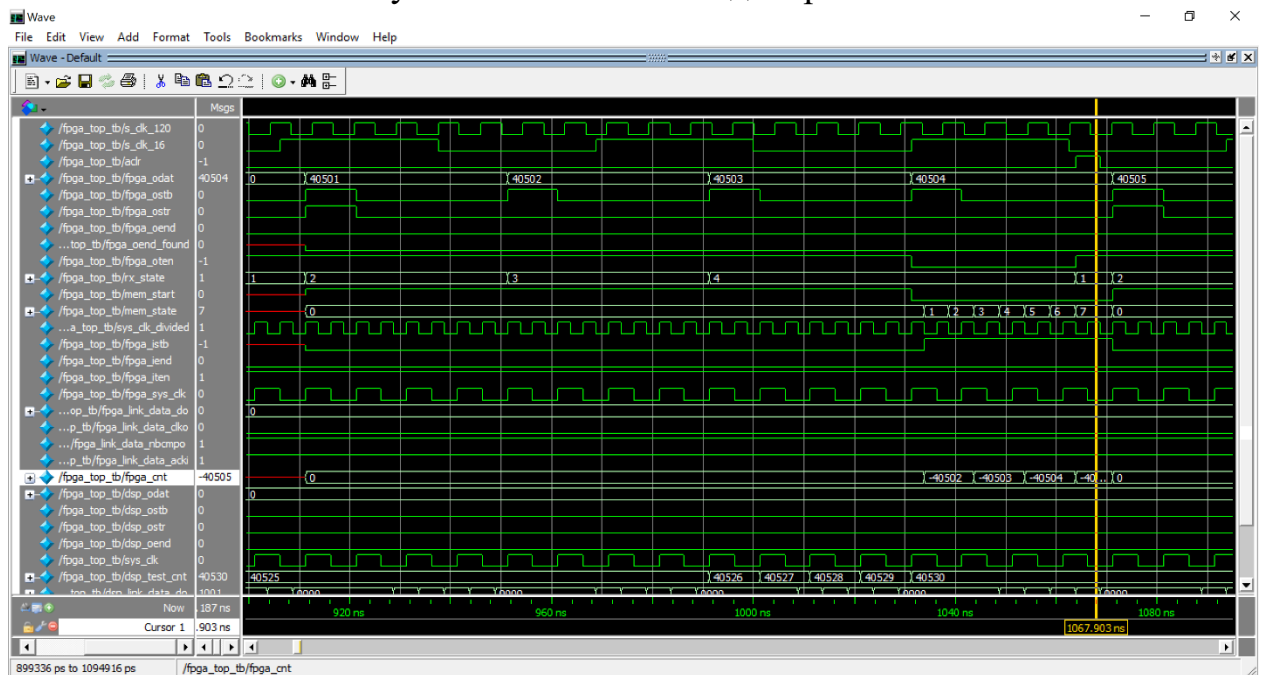


Рисунок Г.2 – Появление первого квадраслова на FPGA-приемнике

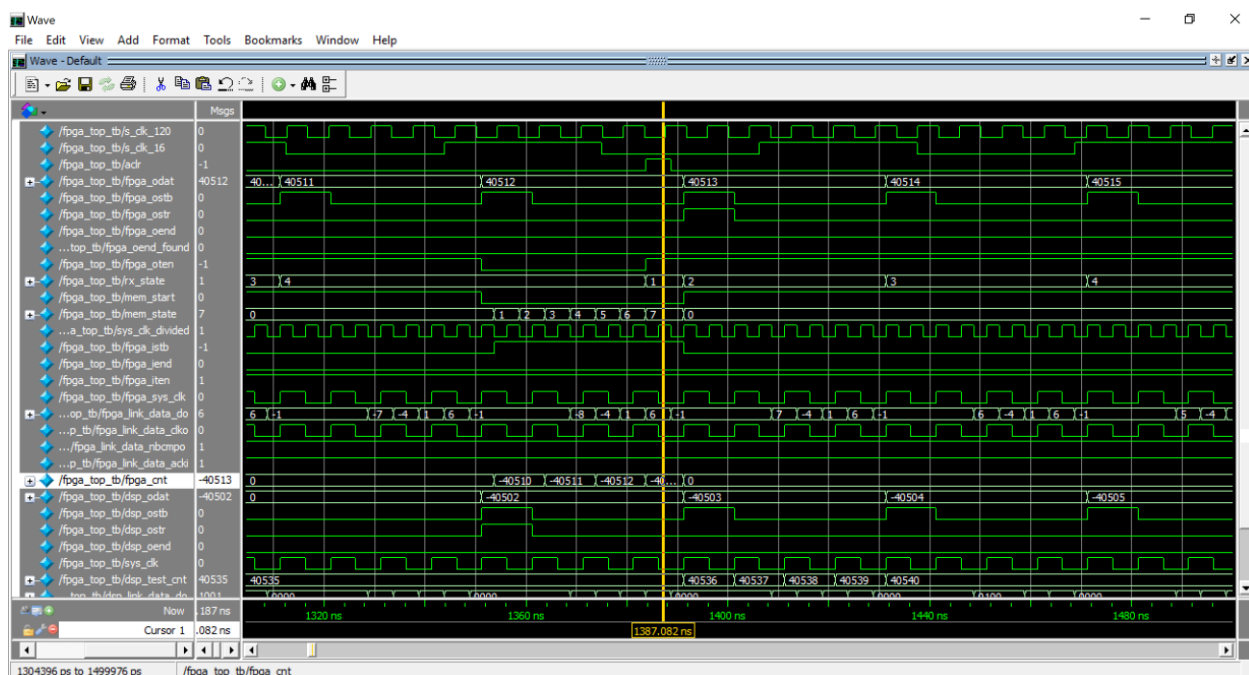


Рисунок Г.3 – Начало выдачи кадров слов на имитации DSP-приемника

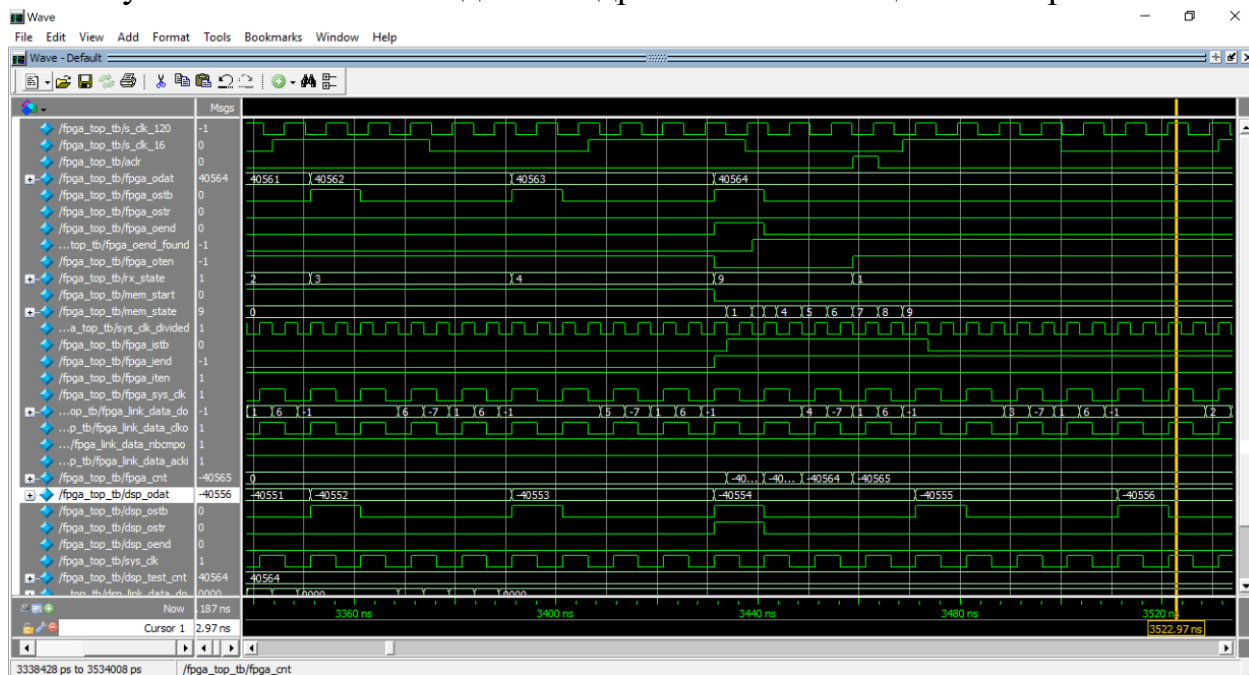


Рисунок Г.4 – Конец приема FPGA-приемника

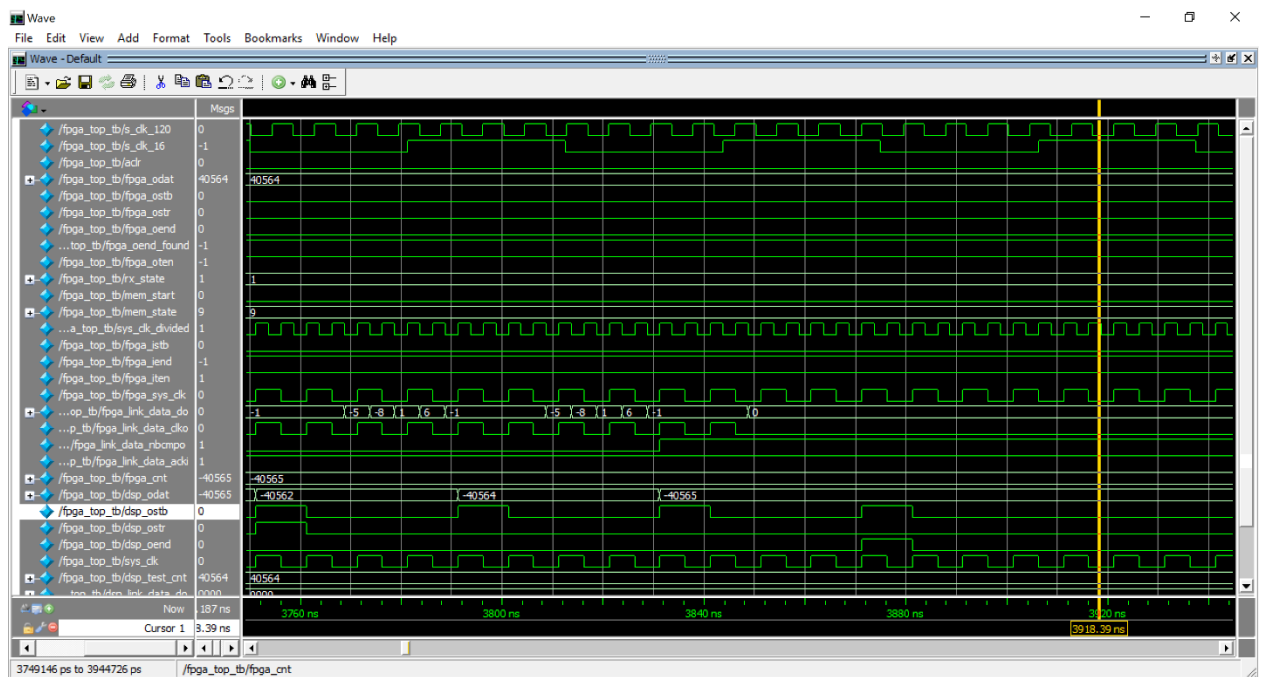


Рисунок Г.5 – конец приема имитации DSP-приемника

ПРИЛОЖЕНИЕ Д. Назначение контактов модуля сопряжения с Link

Таблица Д.1 – Назначение контактов модуля сопряжения с Link

Контакт разъема	Цепь на схеме	Номер pin ПЛИС	Сигнал Verilog	Сигнал	
J10.1	GPIO0	L13			
J10.2	GPIO1	L16		DI[3]	DI 3 n
J10.3	GPIO2	L15	gpio[2]		DI 3 p
J10.4	GPIO3	K16			
J10.5	GPIO4	P16		CLKI	CLKI n
J10.6	GPIO5	R16	gpio[5]		CLKI p
J10.7	GPIO6	N16		CLKO	CLKO n
J10.8	GPIO7	N15	gpio[7]		CLKO p
J10.9	GPIO8	N14			
J10.10	GPIO9	P15			
J10.11	VCC_5V_G PIO	---			
J10.12	GND	---			
J10.13	GPIO10	N8	gpio[10]	ACKO	ACKO
J10.14	GPIO11	P8	gpio[11]	ACKI	ACKI
J10.15	GPIO12	M8	gpio[12]	BCMPI	BCMPI
J10.16	GPIO13	L8		BCMPO	BCMPO
J10.17	GPIO14	R7	gpio[14]	DO[0]	DO 0 p
J10.18	GPIO15	T7	gpio[13]		DO 0 n
J10.19	GPIO16	L7			
J10.20	GPIO17	M7			
J10.21	GPIO18	R6	gpio[18]	DO[1]	DO 1 p
J10.22	GPIO19	T6			DO 1 n
J10.23	GPIO20	T2			
J10.24	GPIO21	M6			
J10.25	GPIO22	R5	gpio[22]	DO[2]	DO 2 p
J10.26	GPIO23	T5			DO 2 n
J10.27	GPIO24	N5	gpio[24]	DO[3]	DO 3 p
J10.28	GPIO25	N6			DO 3 n
J10.29	VCC_3.3V	---			
J10.30	GND	---			
J10.31	GPIO26	R4			
J10.32	GPIO27	T4			
J10.33	GPIO28	N3	gpio[28]	DI[0]	DI 0 p
J10.34	GPIO29	P3			DI 0 n
J10.35	GPIO30	R3	gpio[30]	DI[1]	DI 1 p
J10.36	GPIO31	T3			DI 1 n
J10.37	GPIO32	P6			
J10.38	GPIO33	P2	gpio[33]	DI[2]	DI 2 p
J10.39	GPIO34	P1			DI 2 n
J10.40	GPIO35	R1			

ПРИЛОЖЕНИЕ Е. RTL-диаграммы модуля сопряжения с Link

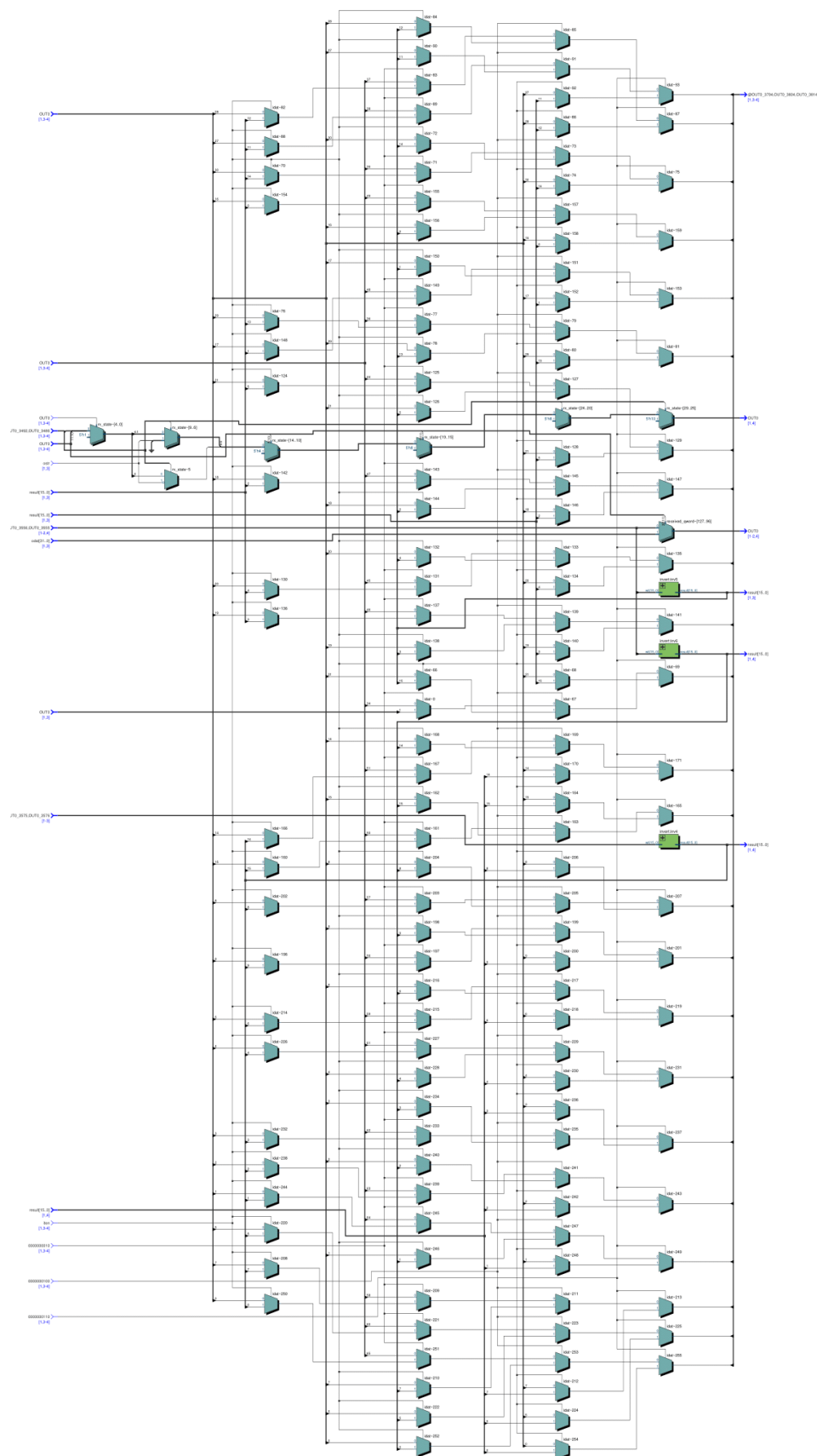


Рисунок Е.1 – Начало RTL-диаграммы

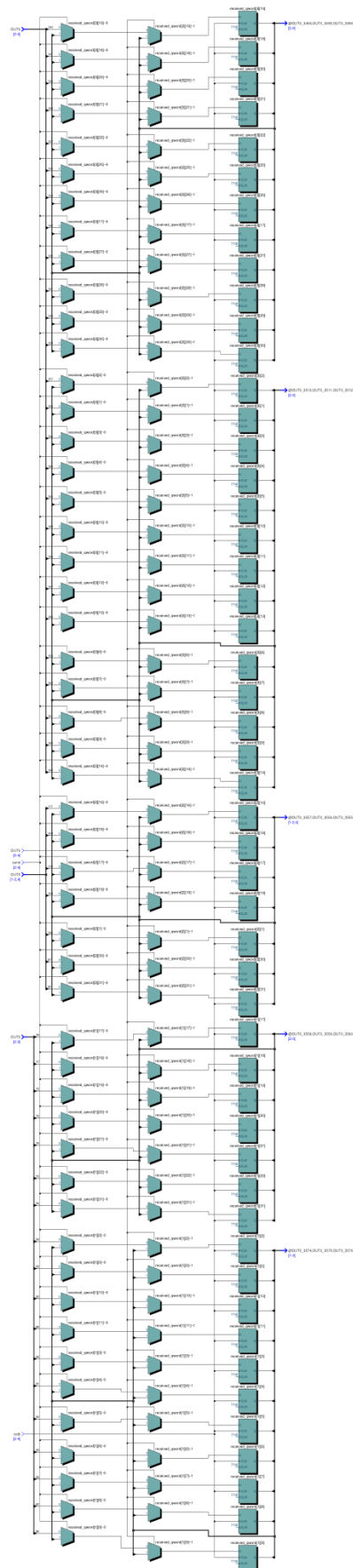


Рисунок Е.2 – Продолжение RTL-диаграммы, регистры для принятого квадрослова

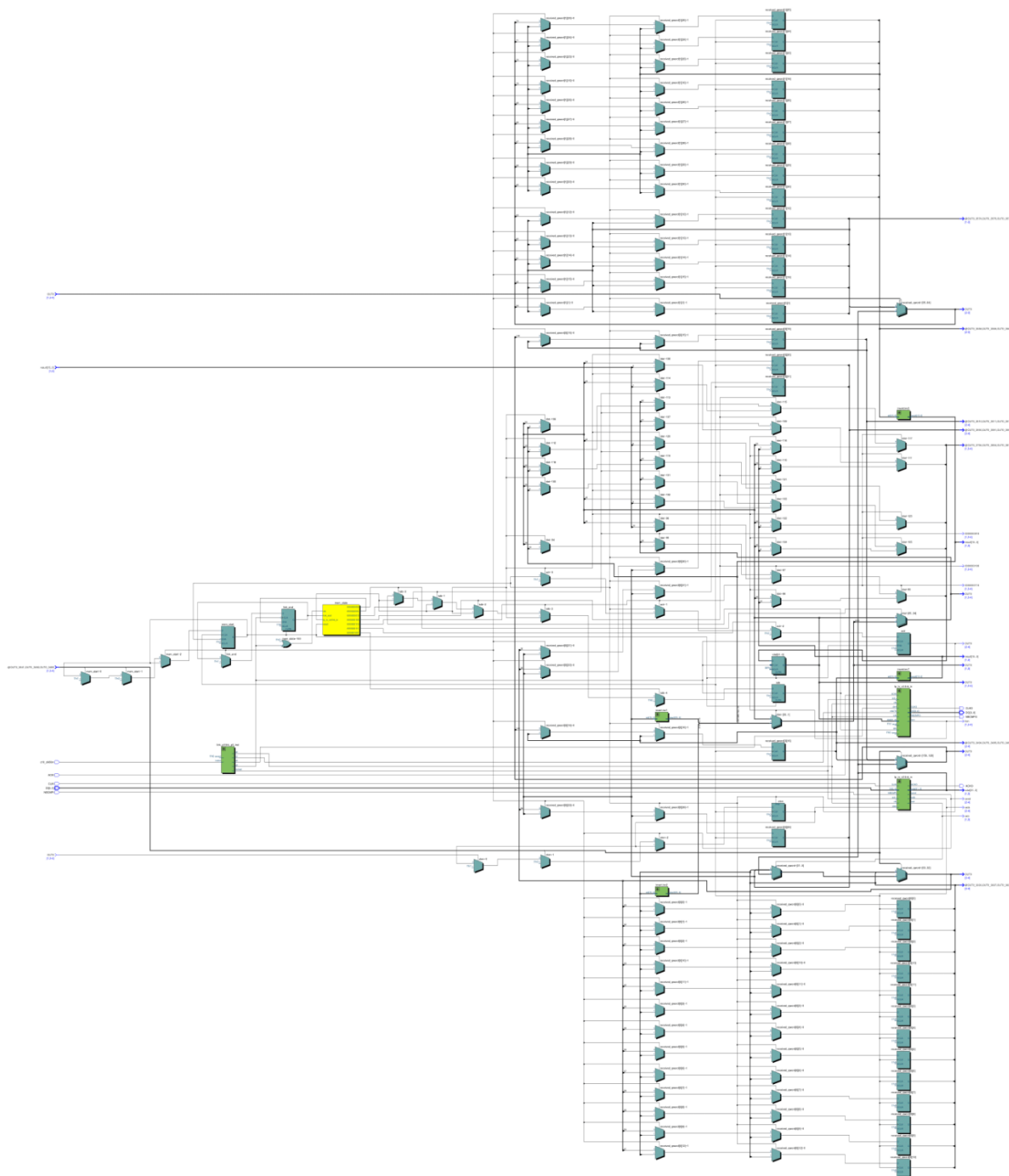


Рисунок Е.3 – Продолжение RTL-диаграммы, модули Link-порта

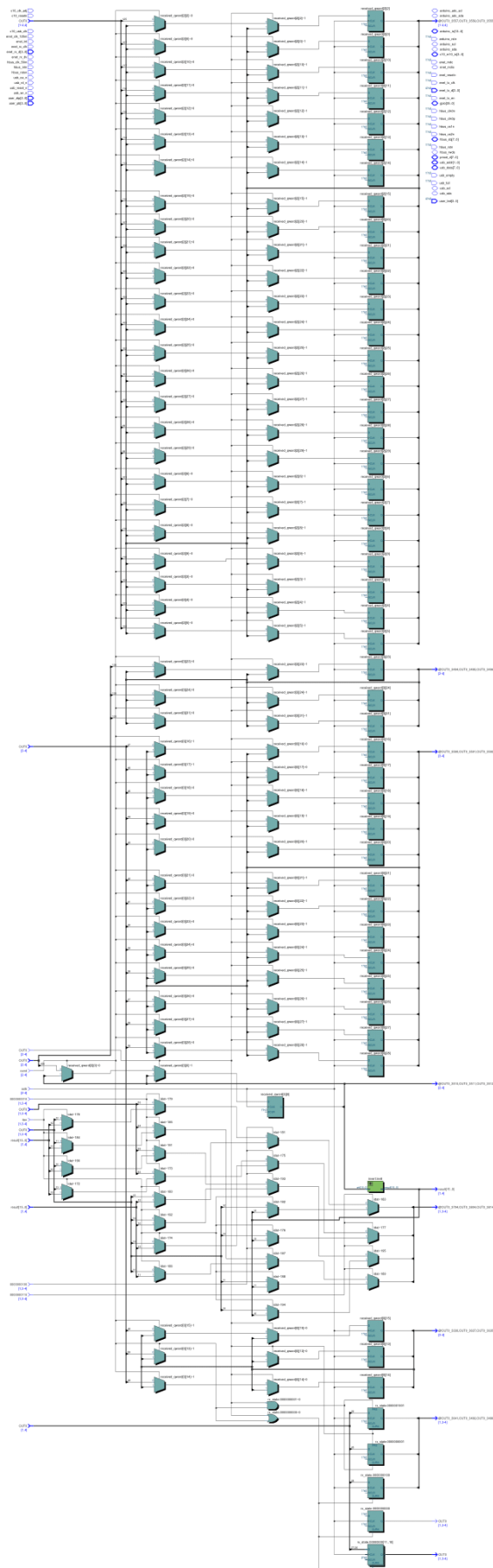


Рисунок Е.4 – Окончание RTL-диаграммы